

Day 1

Today

- ▶ Course Overview
- ▶ ML/DL/AI Review
- ▶ Python Intro

To Do

- ▶ Discussion 1 on Canvas (self-intro, Jan 29th)
- ▶ Join MS Teams Group (join code is: **w87ccaf**)
- ▶ (Optional) Pre-class poll →

<https://www.menti.com/alem7v58cemh>

OR www.menti.com w/ code 3249 1067

OR

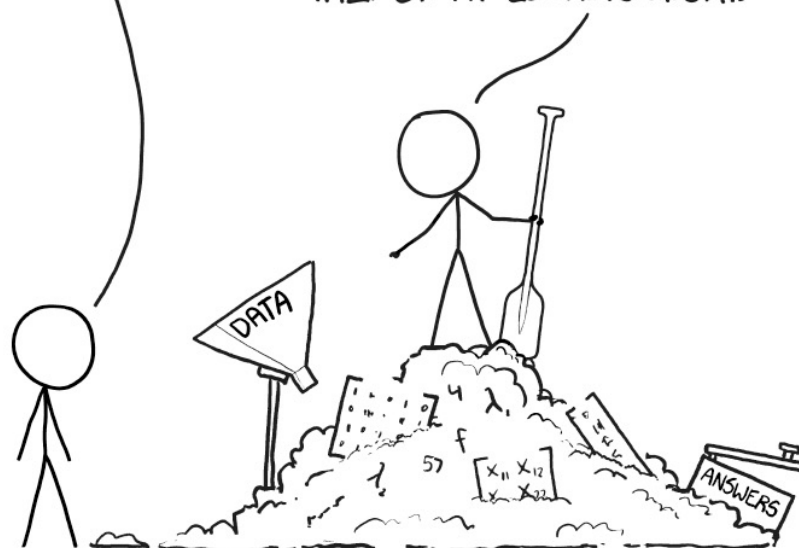


THIS IS YOUR MACHINE LEARNING SYSTEM?

YUP! YOU POUR THE DATA INTO THIS BIG PILE OF LINEAR ALGEBRA, THEN COLLECT THE ANSWERS ON THE OTHER SIDE.

WHAT IF THE ANSWERS ARE WRONG?

JUST STIR THE PILE UNTIL THEY START LOOKING RIGHT.



Jan 20, 2026

New!

AEROSPACE AND ENGINEERING SCIENCES
LEARNING CENTER

**OPEN
HOUSE**

Building a STEM Community

Center for Advanced
STEM Education

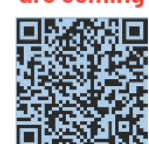
**Wednesday,
January 21, 2026**

Stop by between
1:00 p.m. - 3:00 p.m.

AES 438 STEM Learning Center

- ✓ Learn About CASE Events
- ✓ Food
- ✓ Giveaways

Let us know you
are coming



<https://tinyurl.com/4evf5a7s>

Visit us:
maudenvr.edu/advanced-stem-education/

**MSU
DENVER** Center for Advanced
STEM Education

DSML 4220: Day 1

Introductions

Who am I?

Steve Geinitz (ST-eve GUY-knits, he/him/his)

- ▶ MSU alum in CS and Math
- ▶ former software developer at a small startup (gaming/gambling)
- ▶ graduate studies in CS (ML) and Applied Statistics
- ▶ more recently Data Scientist/Researcher at Facebook, eBay (ads)
- ▶ research interests in Machine/Statistical Learning and Education



Who are you?

Right now (and optional)

- ▶ Poll at <https://www.menti.com/alem7v58cemh>
OR www.menti.com w/ code 3249 1067 OR



Later (and for points, i.e., not optional)

- ▶ Discussion 1 on Canvas (self-intro, due Jan 29th)

Course Intro: Syllabus, Policies, etc.

Let's go to:

Canvas → Modules

1. Course Overview
2. Syllabus
3. Course Grading Policies
4. MS Teams (join code is: **w87ccaf**)
5. Student Resource Guide

Course Intro: Syllabus, Policies, etc.

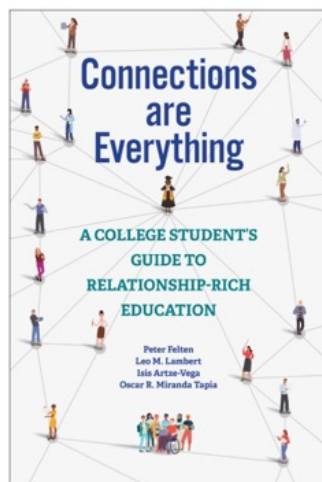
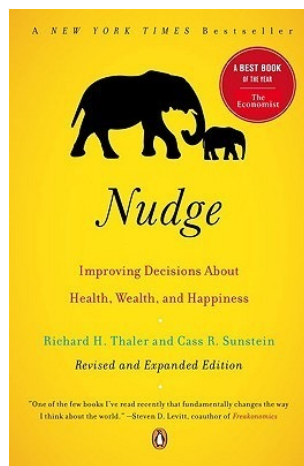


Be sure to visit Canvas on a regular basis!
(past data show Canvas page views correlate positively with final grades)

Course Overview: Participation

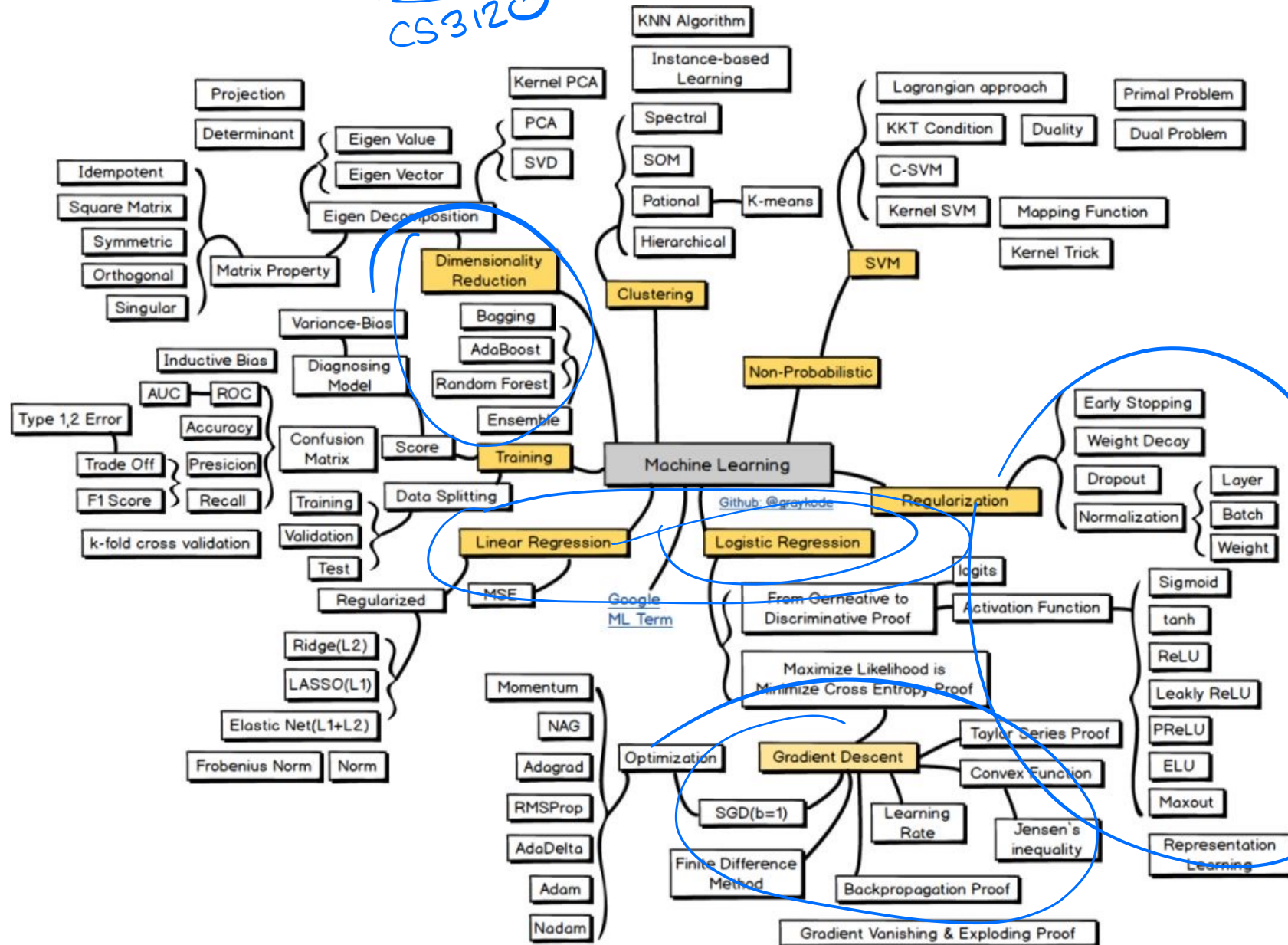
Being present is more important than ever

- attendance not officially taken, but short spontaneous quizzes/polls given during class recognized are counted as a part of Participation grade
- Full quizzes also to be given during class and will have a 20% bonus for collaborating with someone else, but must be present to be eligible (there will be chance to take the quiz later outside of class, although without bonus)
- your future career success depends on you being able to communicate well with other people, and college is a good place to learn/improve this skill
- [new MSU policy](#) is now to automatically remove anyone that is 'missing' first two weeks of class (so that you can receive a refund rather than having to withdraw from the course later, and not lose all of your tuition fee)

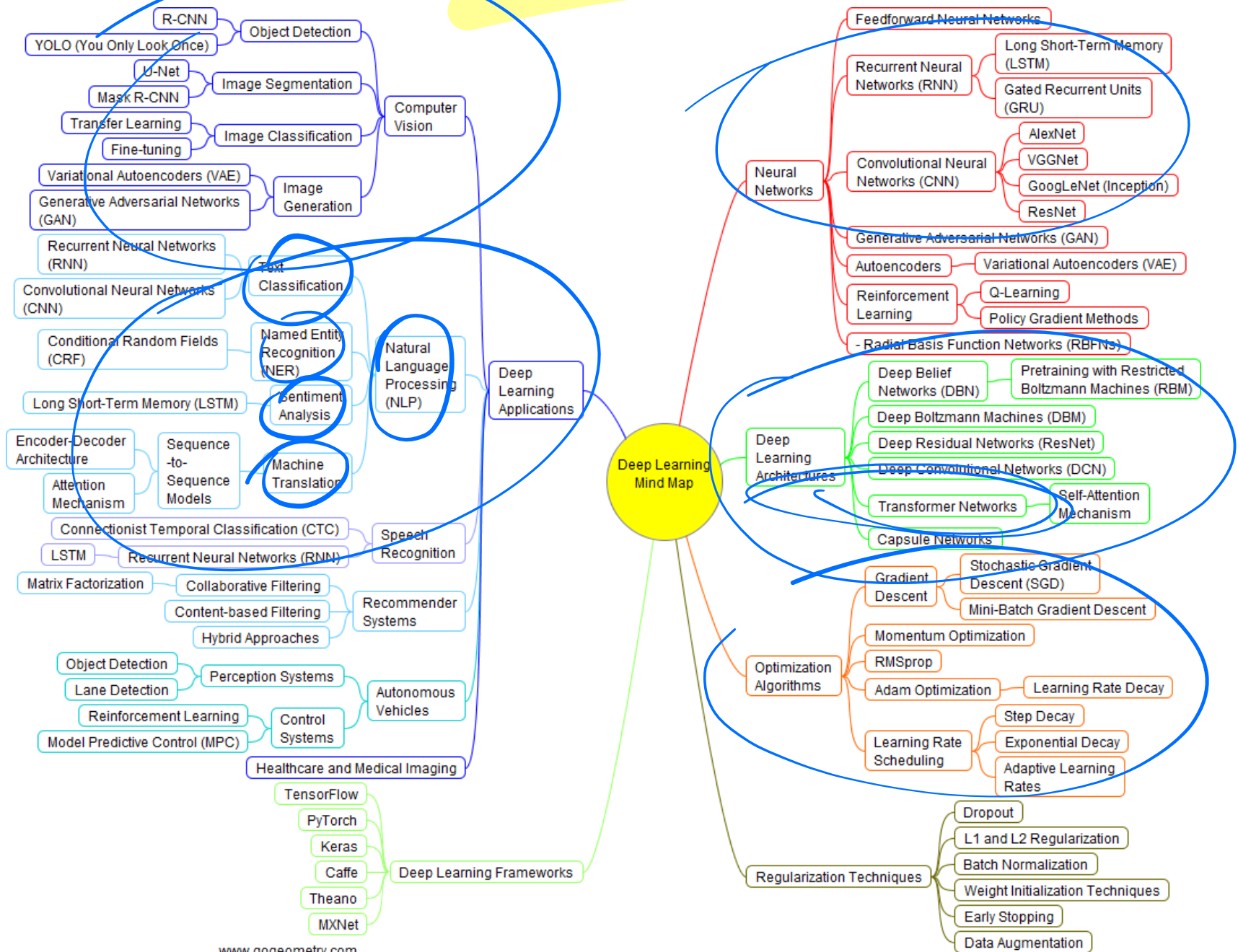


Course Intro: A Machine Learning Mind Map

CS3120



Course Intro: A Deep Learning Mind Map



www.gogeometry.com

Machine Learning

(review)

Review: What is an algorithm?

al·go·rithm | 'algə,riTHəm |



Review: What is an algorithm?

al·go·rithm | 'algə,riTHəm |

- ▶ finite sequence of well-defined, computer-implementable instructions, often to solve a class of problems or perform a computation
- ▶ always unambiguous and used as specifications for performing calculations, data processing, automated reasoning, and other tasks

Examples?

Sorting

shortest path

Review: What is a machine learning?

machine learning*



Review: What is a machine learning?

machine learning*

- ▶ study of computer algorithms that improve automatically through experience and by the use of data (seen as a part of artificial intelligence)
- ▶ algorithms that build a model based on sample data (i.e. "training data") in order to make predictions or decisions without being explicitly programmed to do so – this enables a wide variety of applications, such as in medicine, email filtering, and computer vision, where it is difficult or unfeasible to develop conventional algorithms to perform the needed tasks
- ▶ use and development of computer systems able to learn and adapt without following explicit instructions, by using algorithms and statistical models to analyze and draw inferences from patterns in data

Review: Typical Program/Algo vs ML Program

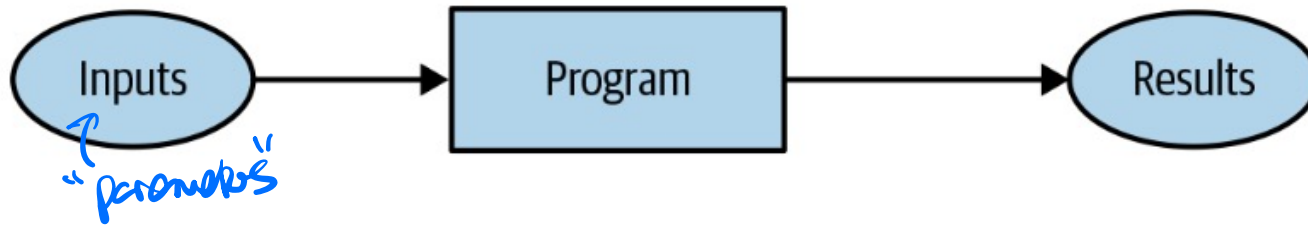
Q: How does a typical computer program/algorithm work?

Q: How does an ML program work?

Review: Typical Program/Algo vs ML Program

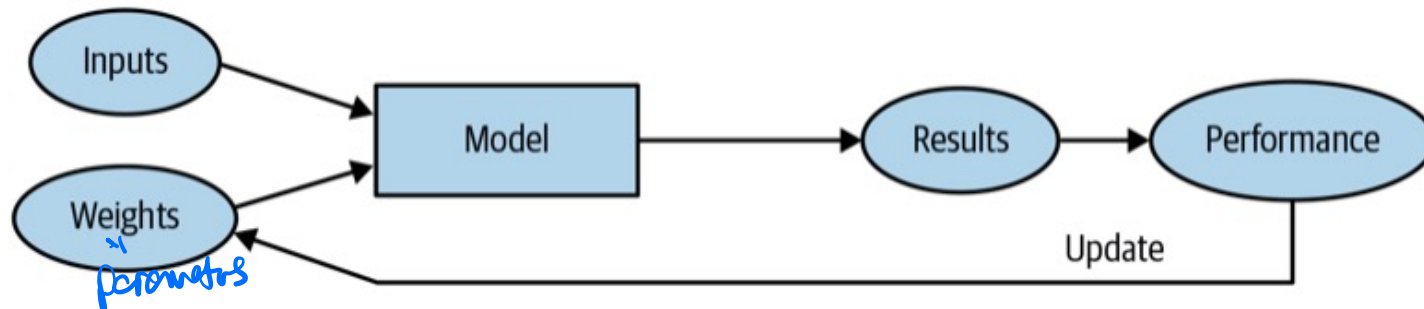
Q: How does a typical computer program/algorithm work?

A:



Q: How ~~does~~ ^{is} an ML program work? ^{trained & created?}

A:



Review: Typical Program/Algo vs ML Program

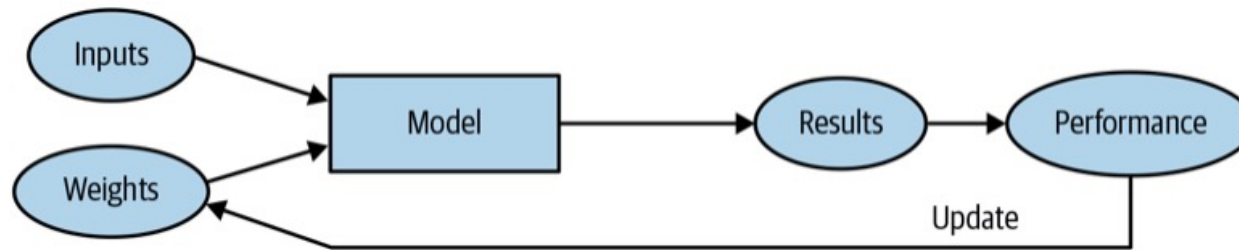
Q: How does a typical computer program/algorithm work?

A:



Q: How is ~~does~~ an ML program trained ~~work~~?

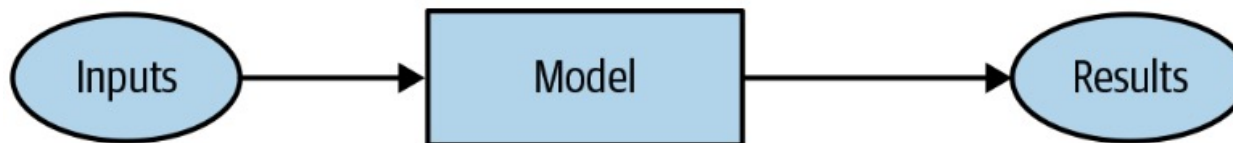
A:



training

Q: How does an ML program work (after its trained)?

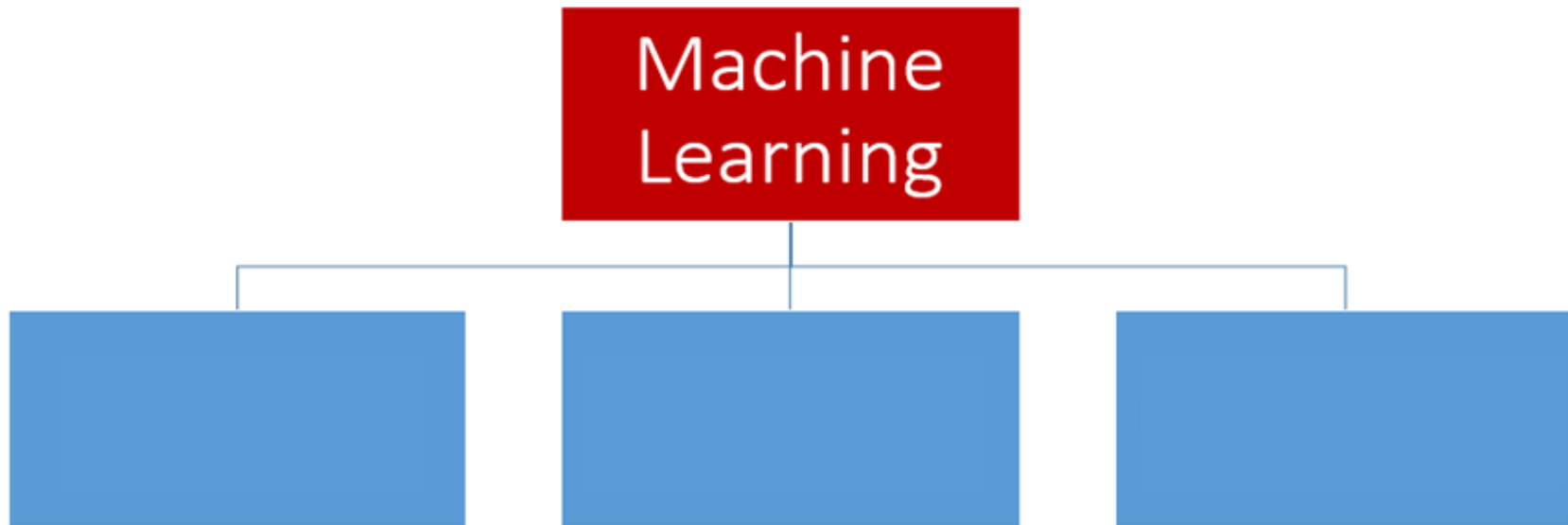
A:



prediction or inference

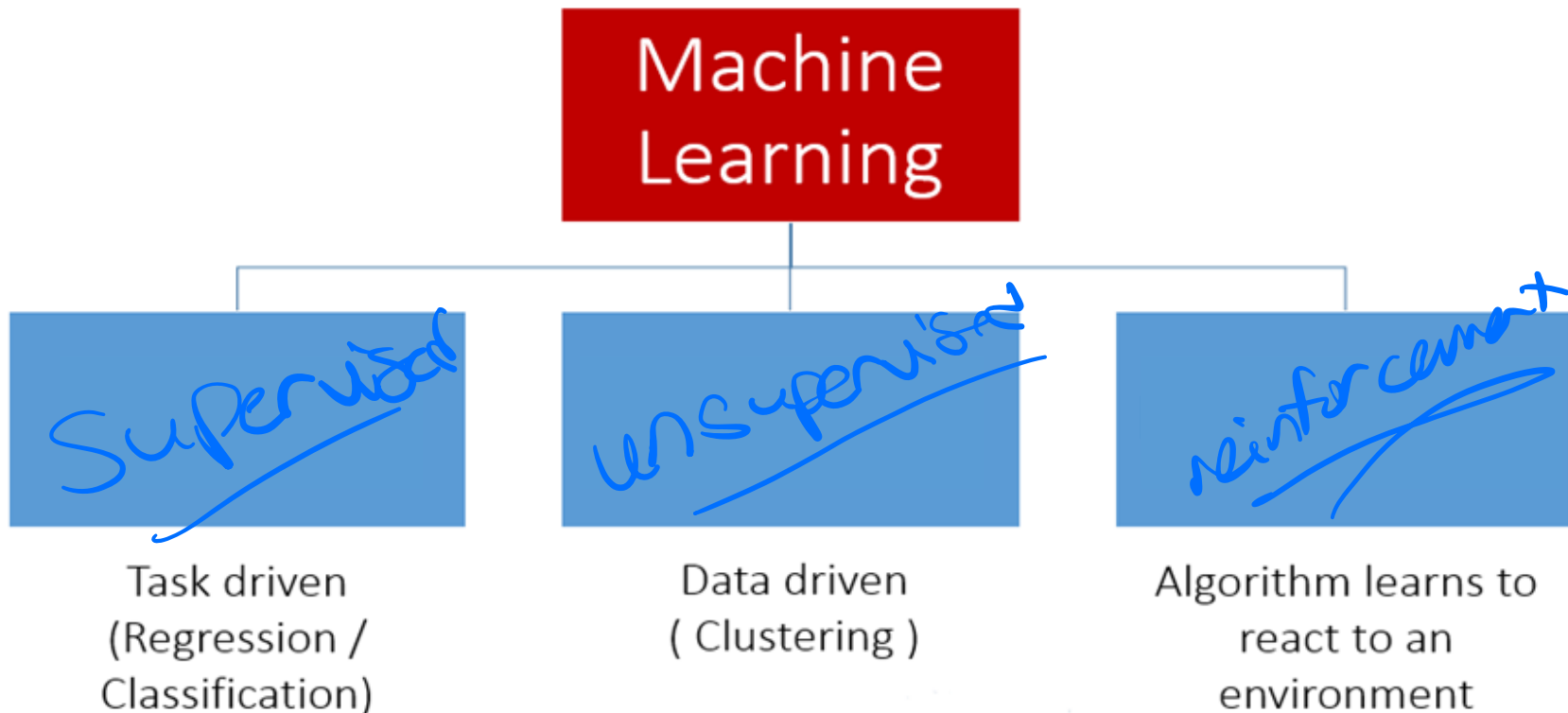
Review: Machine Learning

Q: What are the major types/classes of machine learning?
(i.e. what words belong in the blue boxes?)



Review: Machine Learning

Q: What are the major types/classes of machine learning?
(i.e. what words belong in the blue boxes?)



Review: What is deep learning?

deep learning*



Review: What is deep learning?

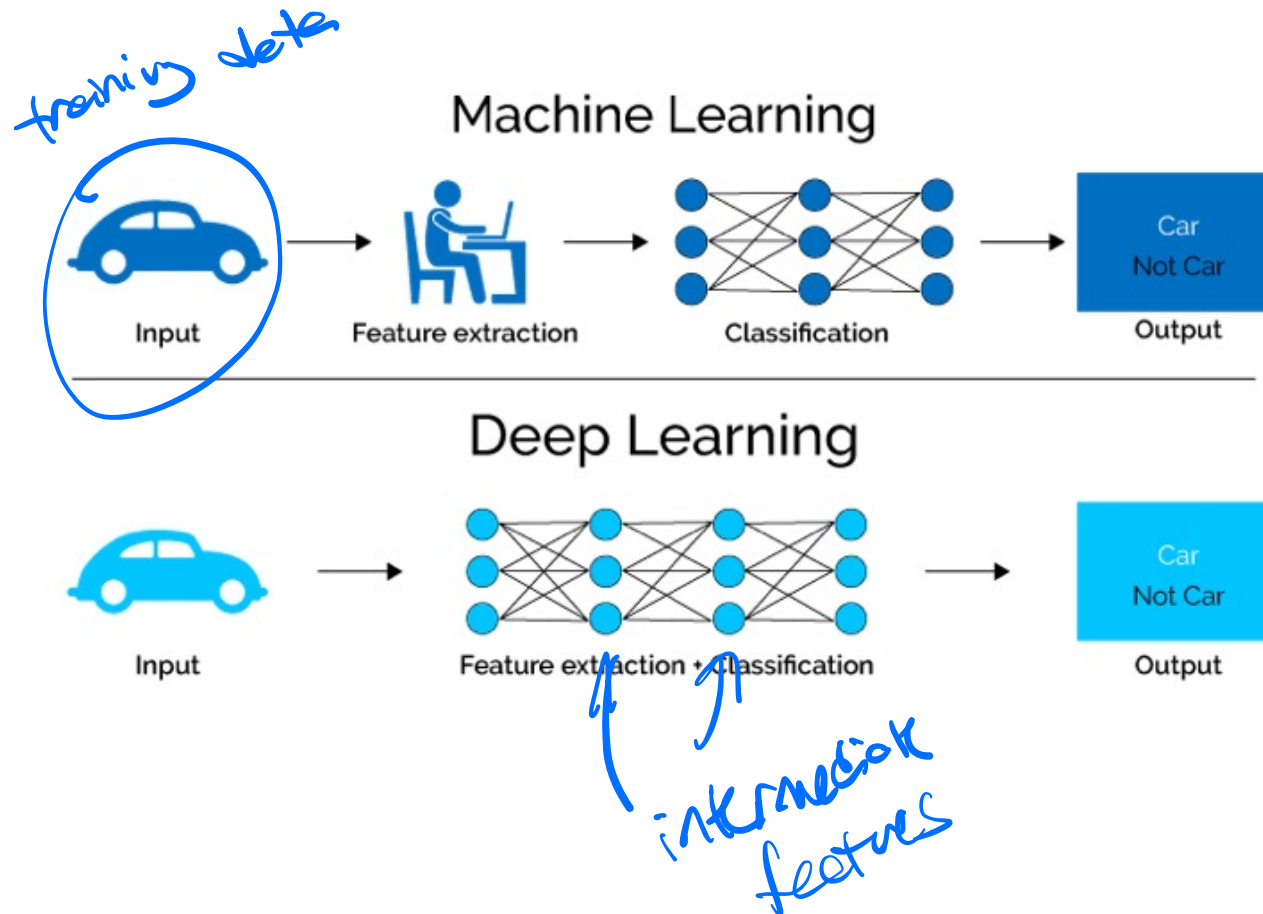
deep learning*

- ▶ part of a broader family of machine learning methods based on artificial neural networks with representation learning; learning can be supervised, semi-supervised or unsupervised
- ▶ "deep" refers to use of multiple layers in the ^{neural} network
 - ▶ early work showed a linear perceptron cannot be a universal classifier, but a network with a nonpolynomial activation function with one hidden layer of unbounded width can
 - ▶ additional layers allows for network to **learn features** (or, representations) of the data, rather than features needing to be explicitly/manually created and provided as input

Review: What is deep learning?

deep learning*

- ▶ part of a broader family of machine learning methods based on artificial neural networks with representation learning; learning can be supervised, semi-supervised or unsupervised



Review: What is Artificial Intelligence?

artificial intelligence*



Review: What is Artificial Intelligence?

artificial intelligence*

- ▶ systems/machines able to mimic the "cognitive" functions associated with the human mind, such as "learning" and "problem solving"

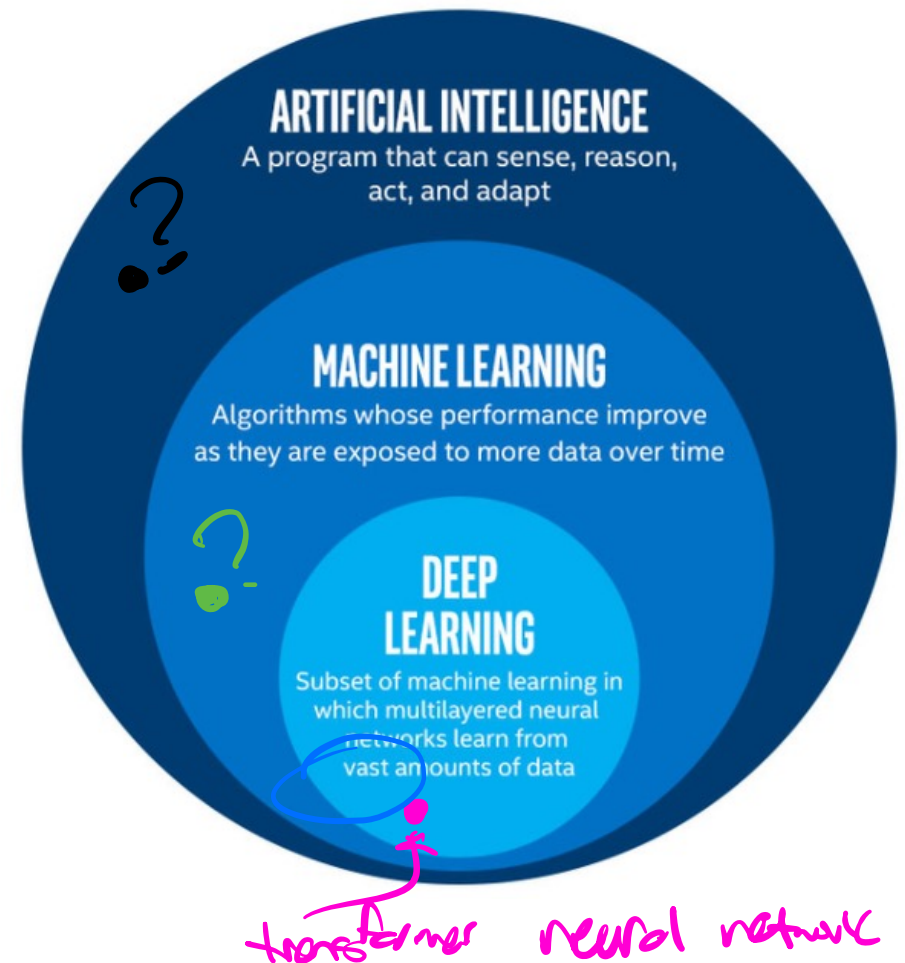
Q: How are ML, DL, and AI related to one another?

Review: What is Artificial Intelligence?

artificial intelligence*

- ▶ systems/machines able to mimic the "cognitive" functions associated with the human mind, such as "learning" and "problem solving"

Q: How are ML, DL, and AI related?



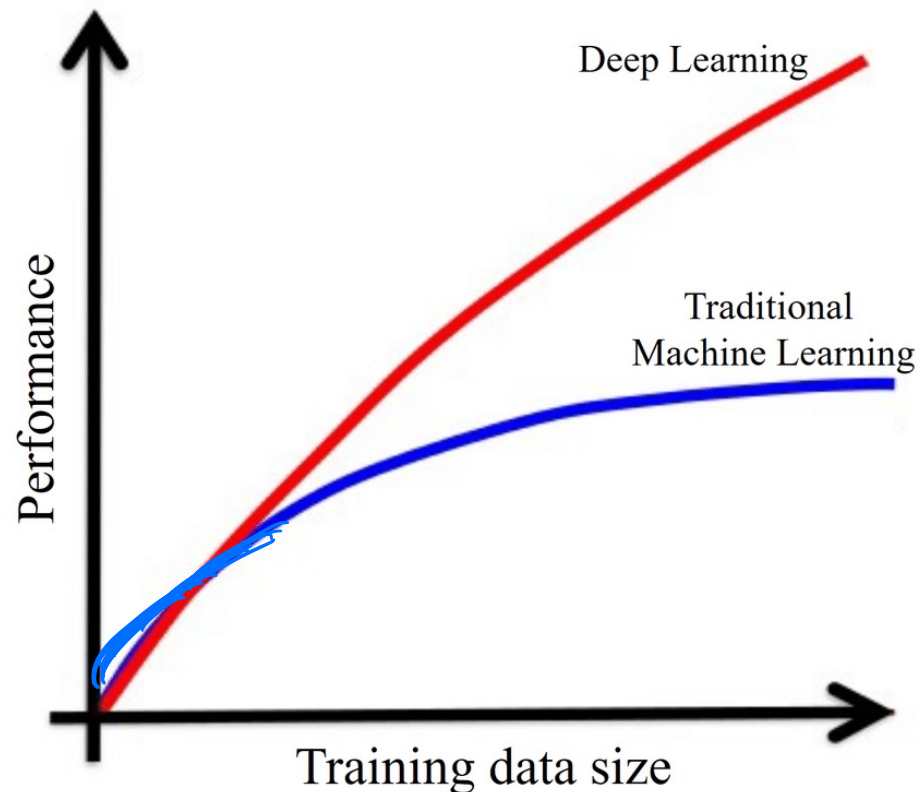
Review: Deep Learning vs 'Regular' ML

Q: Why "*vast amounts of data*"?

Review: Deep Learning vs 'Regular' ML

Q: Why "*vast amounts of data*"?

A: Because deep learning models can learn complex patterns from large datasets (and they typically need a large amount of data to outperform a typical ML model)



Review: Deep Learning vs 'Regular' ML

Q: Where are these "*vast amounts of data*" coming from?

Review: Deep Learning vs 'Regular' ML

Q: Where are these "vast amounts of data" coming from?

 Early Investing

Search

EARLY INVESTING

Data Is Eating the World



BY Vin Narayanan

DATE March 23, 2022

SHARE  SHARE  TWEET

 POST  EMAIL

For more than 10 years, “software is eating the world” has been an article of faith among startup investors. Marc

InfoWorld UNITED STATES SOFTWARE DEVELOPMENT CLOUD COMPUTING INSIDER

Home > Artificial Intelligence > Machine Learning

Data is eating the software that is eating the world

The data-driven machine learning algorithms that power AI will not only upend programming, but lower the barriers to AI itself



 By James Kobielus
Contributor, InfoWorld | JUL 28, 2017 3:00 AM PDT

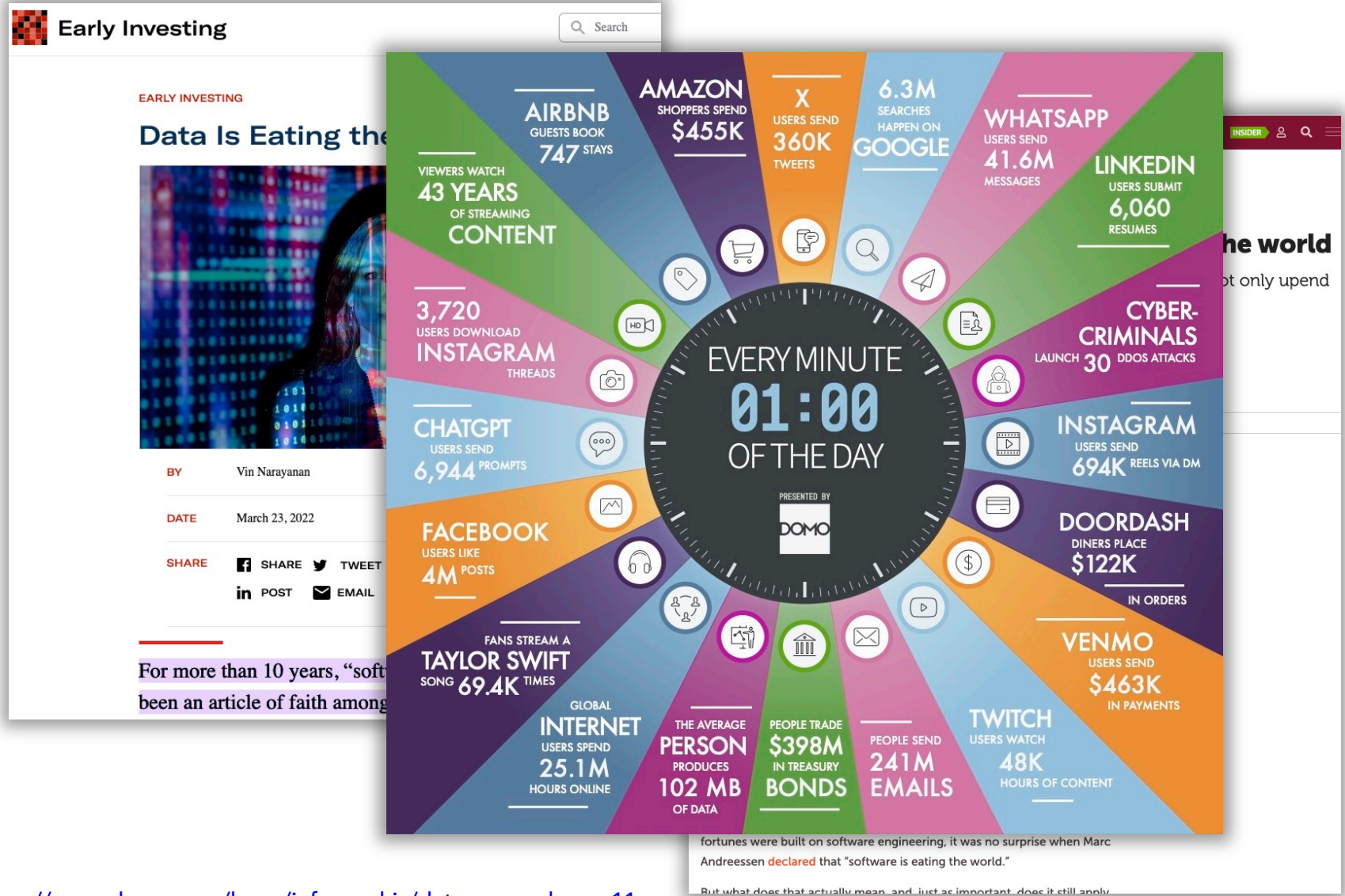


No one doubts that **software engineering** shapes every last facet of our 21st century existence. Given his vested interest in companies whose fortunes were built on software engineering, it was no surprise when Marc Andreessen **declared** that “software is eating the world.”

But what does that actually mean, and just as important, does it still apply

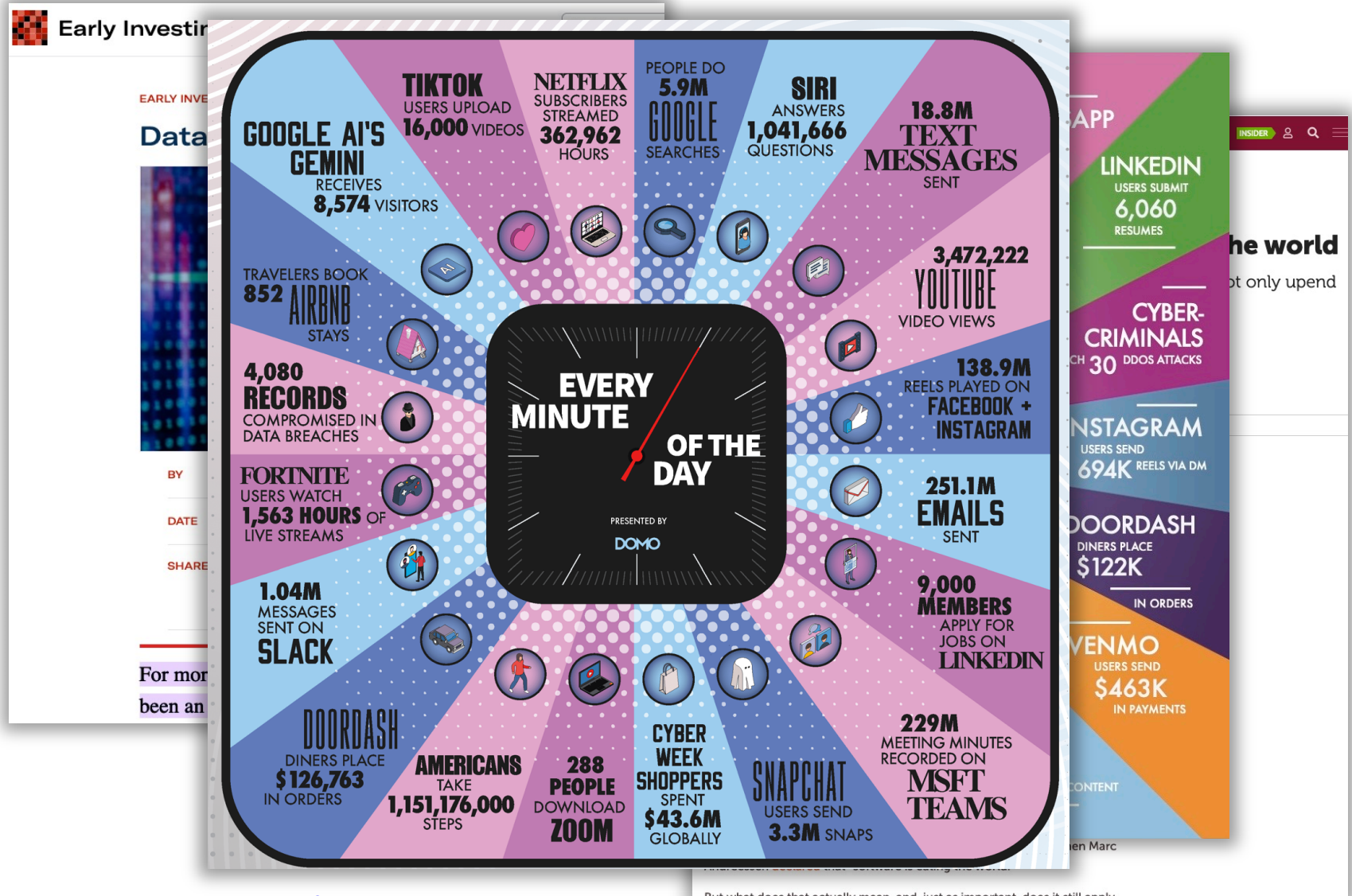
Review: Deep Learning vs 'Regular' ML

Q: Where are these "vast amounts of data" coming from?



Review: Deep Learning vs 'Regular' ML

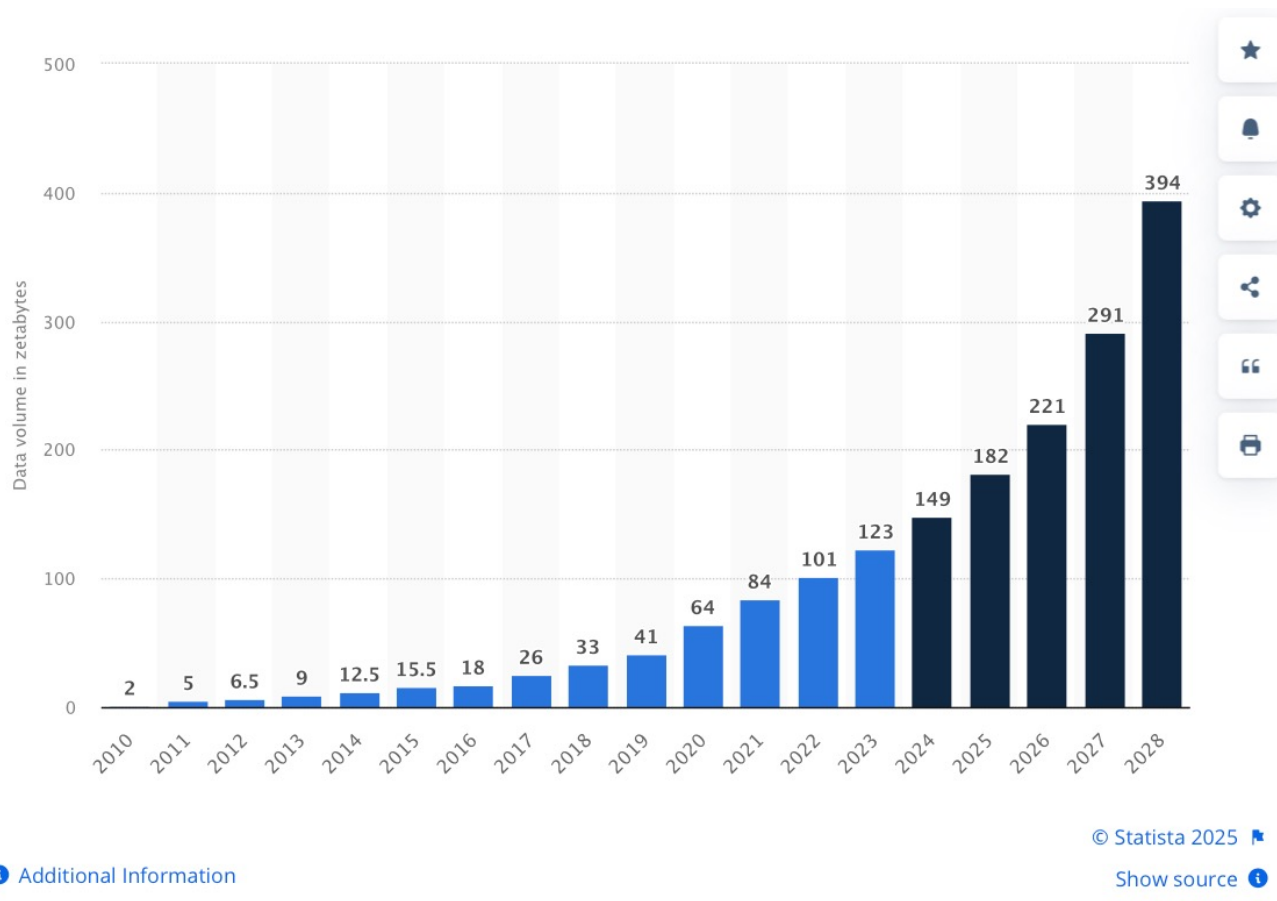
Q: Where are these "vast amounts of data" coming from?



Review: Deep Learning vs 'Regular' ML

Q: Where are these “*vast amounts of data*” coming from?

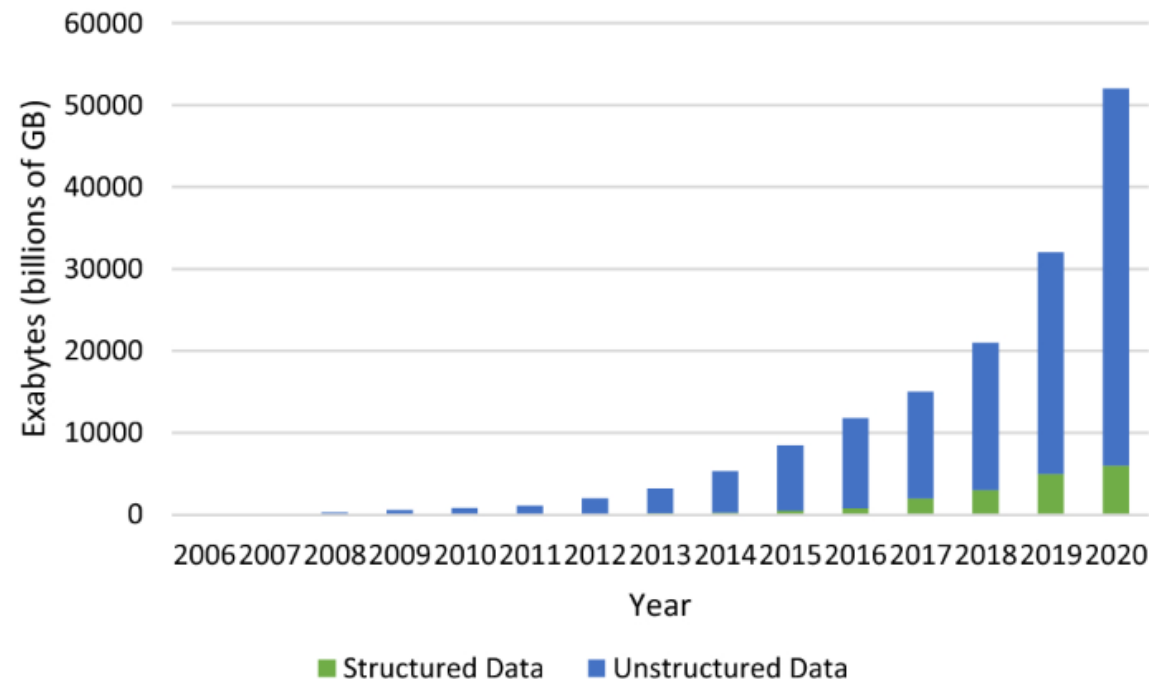
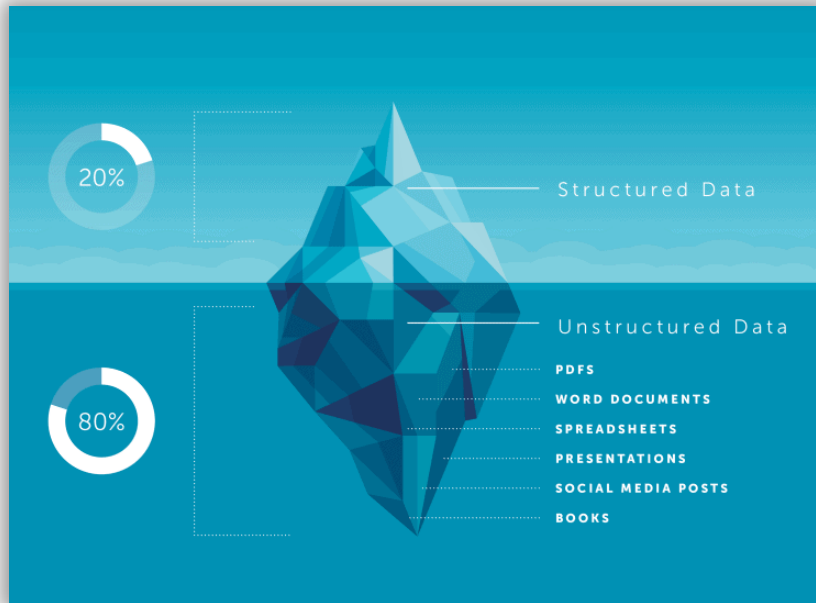
A: In short, every interaction with a device/website/etc. generates data



Review: Deep Learning vs 'Regular' ML

Q: Where are these “*vast amounts of data*” coming from?

A: In short, every interaction with a device/website/etc. generates data and much of it is human communication (so unstructured data)



Review/Intro: Structured vs Unstructured Data

Q: What is the difference between structured and unstructured data?

Review/Intro: Structured vs Unstructured Data

Q: What is the difference between structured and unstructured data?

Structured Data: follows a predefined schema or format (e.g., rows and columns)

- easy to store, search, and analyze using traditional database systems
- examples:
 - spreadsheets of sales data (e.g., date, product, price, etc.)
 - customer databases (e.g., name, age, email)
 - sensor readings (e.g., temperature, humidity)

Review/Intro: Structured vs Unstructured Data

Q: What is the difference between structured and unstructured data?

Structured Data: follows a predefined schema or format (e.g., rows and columns)

- easy to store, search, and analyze using traditional database systems
- examples:
 - spreadsheets of sales data (e.g., date, product, price, etc.)
 - customer databases (e.g., name, age, email)
 - sensor readings (e.g., temperature, humidity)

Unstructured Data: no predefined format or organization

- more complex and difficult to analyze directly
- examples:
 - text documents (emails, article, books, messages, etc.)
 - images and videos
 - audio recordings or speech data

Review/Intro: Structured vs Unstructured Data

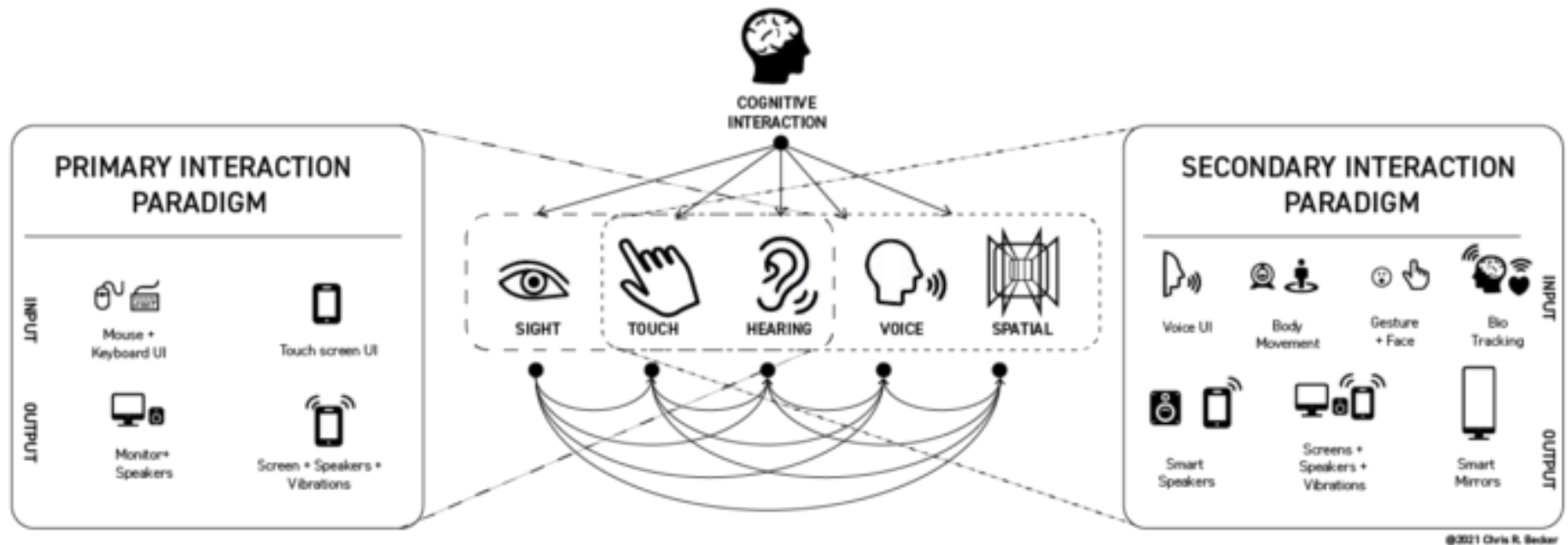
Q: Why else are language and images/vision important or useful?

Review/Intro: Structured vs Unstructured Data

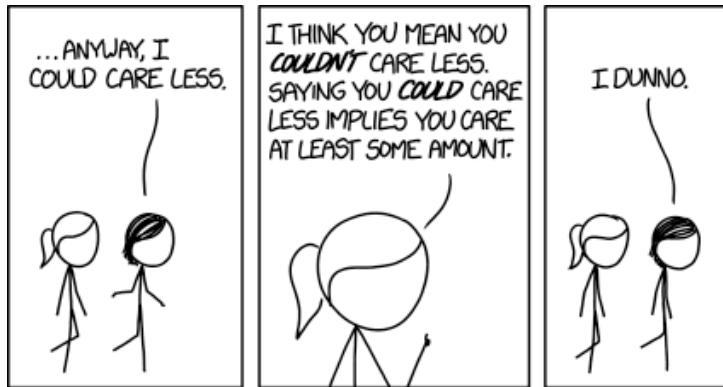
Q: Why else are language and images/vision important or useful?

A: They are increasingly common as human-computer interfaces

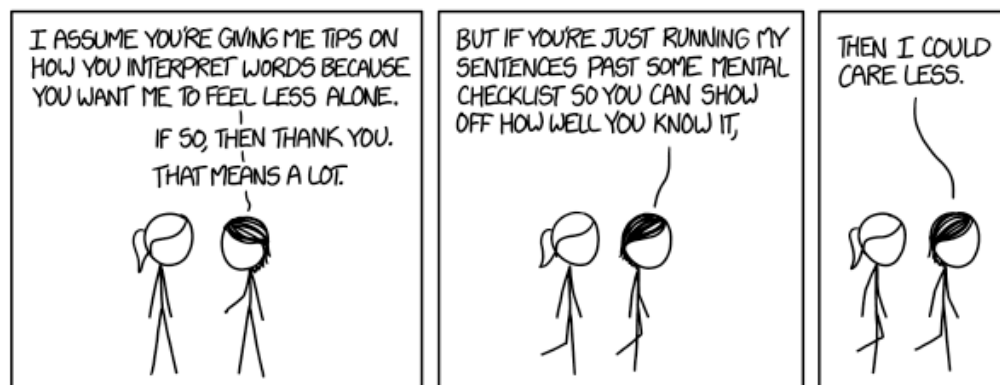
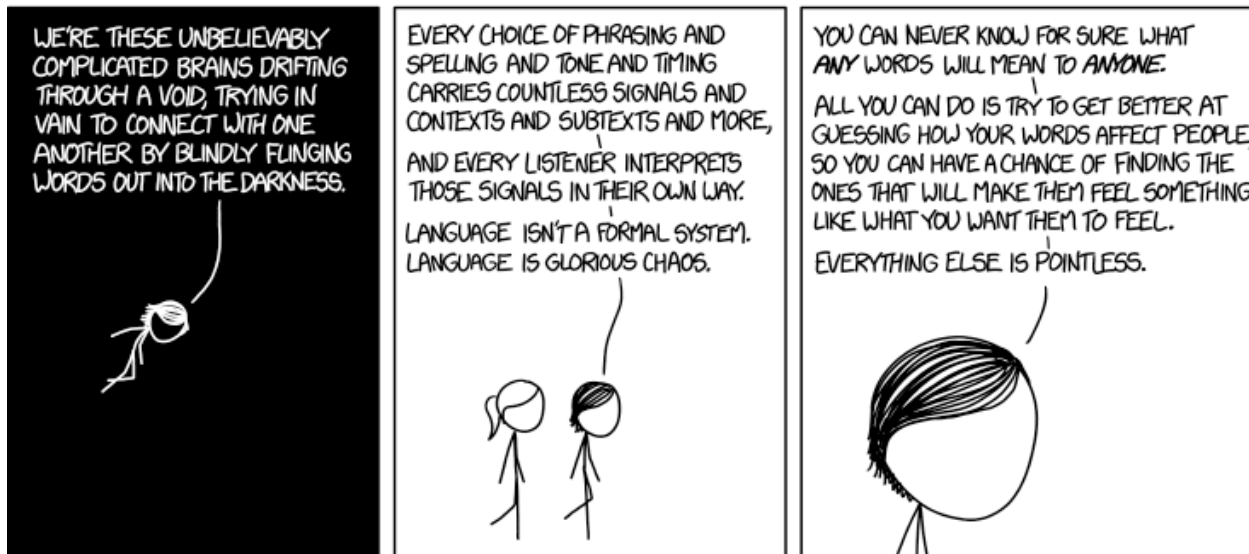
HUMAN INTERACTION WITH A COMPUTER



Review/Intro: Language and Vision



Q: What's so difficult about language and vision?



Review/Intro: Language and Vision

Q: What's so difficult about language and vision?

Language Challenges

- high dimensionality (large vocabulary, complex relationships between words, so feature representation is not easy)
- context (word meanings often depends on context)
- ambiguity/variability (idioms, slang, and diverse grammatical)
- sequential in nature (word order and dependencies not easy to specify)

Review/Intro: Language and Vision

Q: What's so difficult about language and vision?

Language Challenges

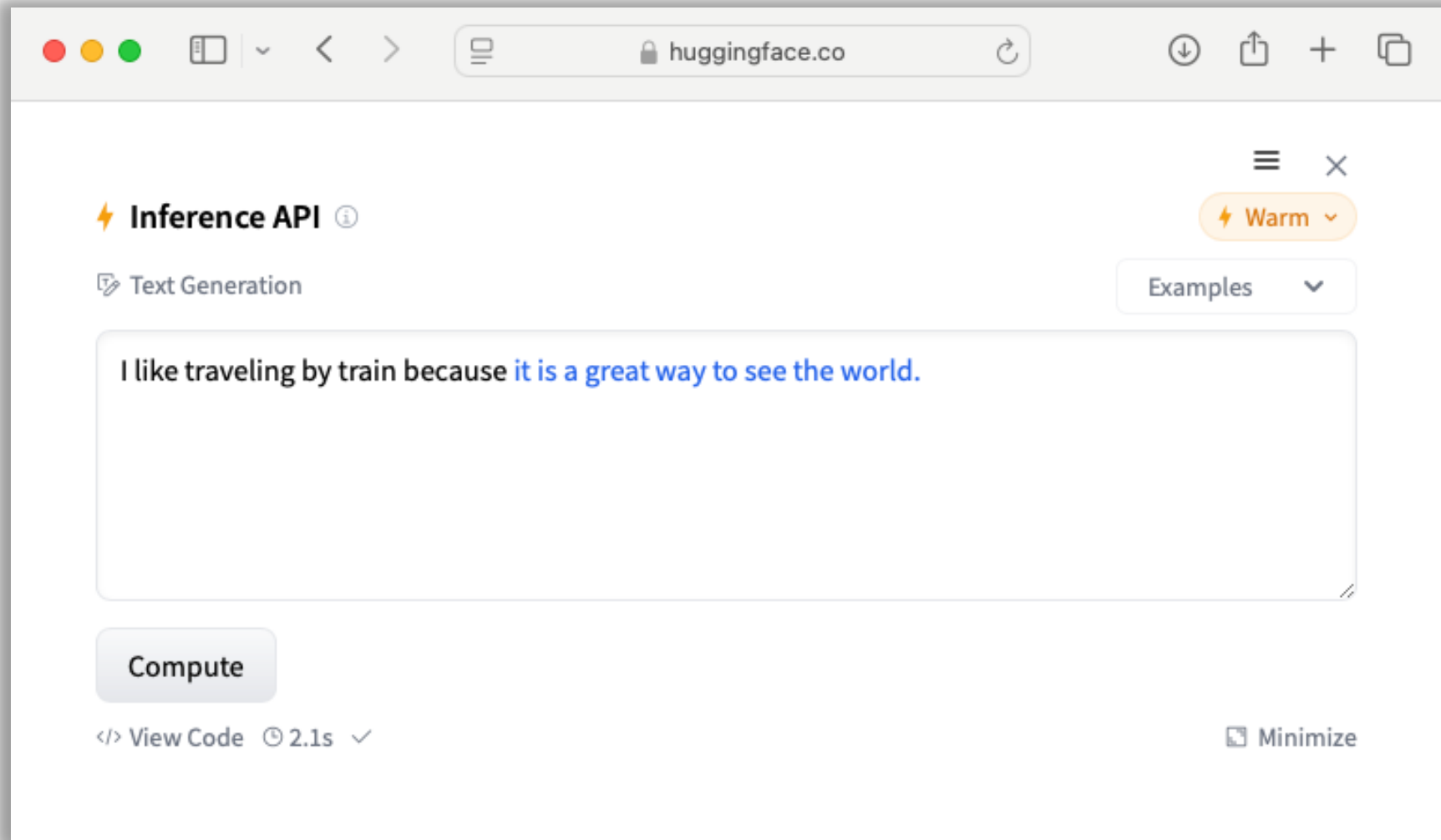
- high dimensionality (large vocabulary, complex relationships between words, so feature representation is not easy)
- context (word meanings often depends on context)
- ambiguity/variability (idioms, slang, and diverse grammatical)
- sequential in nature (word order and dependencies not easy to specify)

Image Challenges

- high dimensionality (typically large array of pixel values, leading to large feature spaces that are computationally expensive to process)
- spatial relationships (relationships between pixels, e.g., edges, shapes, textures, is non-trivial for traditional algorithms)
- lack of invariance (variation in scale, orientation, and lighting, make generalization difficult)

Review/Intro: Some basic NLP applications/tools

Ex 1: Text generation with Llama-3.2-1B



Review/Intro: Some basic NLP applications/tools

Ex 2: Text summarization with BART

The screenshot shows the Hugging Face model card for BART (large-sized model), fine-tuned on CNN Daily Mail. The card is displayed in a web browser window with the URL <https://huggingface.co/facebook/bart-large-cnn>. The card includes a description of the model, its intended uses, and a sample of text summarization. The model is a transformer encoder-decoder (seq2seq) model with a bidirectional (BERT-like) encoder and an autoregressive (GPT-like) decoder. It is pre-trained by (1) corrupting text with an arbitrary noising function, and (2) learning a model to reconstruct the original text. The model is particularly effective when fine-tuned for text generation (e.g. summarization, translation) but also works well for comprehension tasks (e.g. text classification, question answering). This particular checkpoint has been fine-tuned on CNN Daily Mail, a large collection of text-summary pairs. The intended uses & limitations section states that the model can be used for text summarization. The 'How to use' section is partially visible. On the right side of the card, there is a 'Downloads last month' graph showing 3,167,660 downloads. Below the graph, there are tabs for 'Safetensors' and 'Inference API'. The 'Inference API' tab is selected, showing a 'Warm' status. A sample of text summarization is displayed, showing the input text and the summarized output. The input text is: 'The tower is 324 metres (1,063 ft) tall, about the same height as an 81-storey building, and the tallest structure in Paris. Its base is square, measuring 125 metres (410 ft) on each side. During its construction, the Eiffel Tower surpassed the Washington Monument to become the tallest man-made structure in the world, a title it held for 41 years until the Chrysler Building in New York City was finished in 1930. It was the first structure to reach a height of 300 metres. Due to the addition of a broadcasting aerial at the top of the tower in 1957, it is now taller than the Chrysler Building by 5.2 metres (17 ft). Excluding transmitters, the Eiffel Tower is the second tallest free-standing structure in France after the Millau Viaduct.' The summarized output is: 'The tower is 324 metres (1,063 ft) tall, about the same height as an 81-storey building. Its base is square, measuring 125 metres (410 ft) on each side. During its construction, the Eiffel Tower surpassed the Washington Monument to become the tallest man-made structure in the world.'

BART (large-sized model), fine-tuned on CNN Daily Mail

BART model pre-trained on English language, and fine-tuned on CNN Daily Mail. It was introduced in the paper BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension by Lewis et al. and first released in [this repository] (<https://github.com/pytorch/fairseq/tree/master/examples/bart>).

Disclaimer: The team releasing BART did not write a model card for this model so this model card has been written by the Hugging Face team.

Model description

BART is a transformer encoder-decoder (seq2seq) model with a bidirectional (BERT-like) encoder and an autoregressive (GPT-like) decoder. BART is pre-trained by (1) corrupting text with an arbitrary noising function, and (2) learning a model to reconstruct the original text.

BART is particularly effective when fine-tuned for text generation (e.g. summarization, translation) but also works well for comprehension tasks (e.g. text classification, question answering). This particular checkpoint has been fine-tuned on CNN Daily Mail, a large collection of text-summary pairs.

Intended uses & limitations

You can use this model for text summarization.

How to use

Downloads last month
3,167,660

Safetensors

Model size 406M params Tensor type F32

Inference API Warm

Summarization Examples

The tower is 324 metres (1,063 ft) tall, about the same height as an 81-storey building, and the tallest structure in Paris. Its base is square, measuring 125 metres (410 ft) on each side. During its construction, the Eiffel Tower surpassed the Washington Monument to become the tallest man-made structure in the world, a title it held for 41 years until the Chrysler Building in New York City was finished in 1930. It was the first structure to reach a height of 300 metres. Due to the addition of a broadcasting aerial at the top of the tower in 1957, it is now taller than the Chrysler Building by 5.2 metres (17 ft). Excluding transmitters, the Eiffel Tower is the second tallest free-standing structure in France after the Millau Viaduct.

Compute

The tower is 324 metres (1,063 ft) tall, about the same height as an 81-storey building. Its base is square, measuring 125 metres (410 ft) on each side. During its construction, the Eiffel Tower surpassed the Washington Monument to become the tallest man-made structure in the world.

View Code 0.2s Cached example Maximize

Review/Intro: Some basic NLP applications/tools

Ex 2: Text summarization with BART

THE WALL STREET JOURNAL

Home World U.S. Politics **Economy** Business Tech Markets Opinion Books & Arts Real Estate Life & Work

U.S. Home Sales Dropped in July for Sixth Straight Month

Median sale price eased last month to \$403,800 from June record of \$413,800



The housing market has been losing steam as the cost of taking out a mortgage has increased.
PHOTO: LAURA MCDERMOTT FOR THE WALL STREET JOURNAL

By [David Harrison](#) [Follow](#) and [Nicole Friedman](#) [Follow](#)
Updated Aug. 18, 2022 2:05 pm ET

SAVE PRINT TEXT 156

[Listen to article \(7 minutes\)](#) [Queue](#)

U.S. existing home sales fell in July for the sixth straight month, the longest streak of declines in more than eight years, as [higher mortgage rates](#) and a [shortage of homes for sale](#) are cooling this once red-hot market.

Sales of previously owned homes dipped 5.9% in July from the previous month to a seasonally adjusted annual rate of 4.81 million, the National Association of Realtors said Thursday. That was the weakest pace of sales since November 2015, excluding the three-month pandemic-related drop in the spring of 2020. July sales tumbled 20.2% from a year ago.

The drop-off is the latest sign that the formerly booming housing market is stalling out



Summarization Examples

prices. The median home sold in 14 days in July, matching the fastest pace on record, according to NAR.

At the end of July there were 1.31 million homes available, up 4.8% from June, according to NAR, a sign that the market might be starting to loosen up.

“Just in the last 45 days, it’s a whole new market,” said Quentin Dane, a real-estate agent in Raleigh, N.C. The number of new leads coming into his brokerage has dropped by half, he said, and the inventory of homes for sale has increased. “We’re definitely seeing buyers take a breath,” he said.

Almost 16% of home-purchase agreements that were pending in July fell through, the highest level since April 2020, when the pandemic disrupted the market, according to real-estate brokerage Redfin Corp. Some buyers are backing out if sellers won’t renegotiate the price or make requested repairs, and others are worried about a potential recession, Redfin said.

Nicholas and Katie Caracella started house hunting in the

Compute

Computation time on cpu: 7.717 s

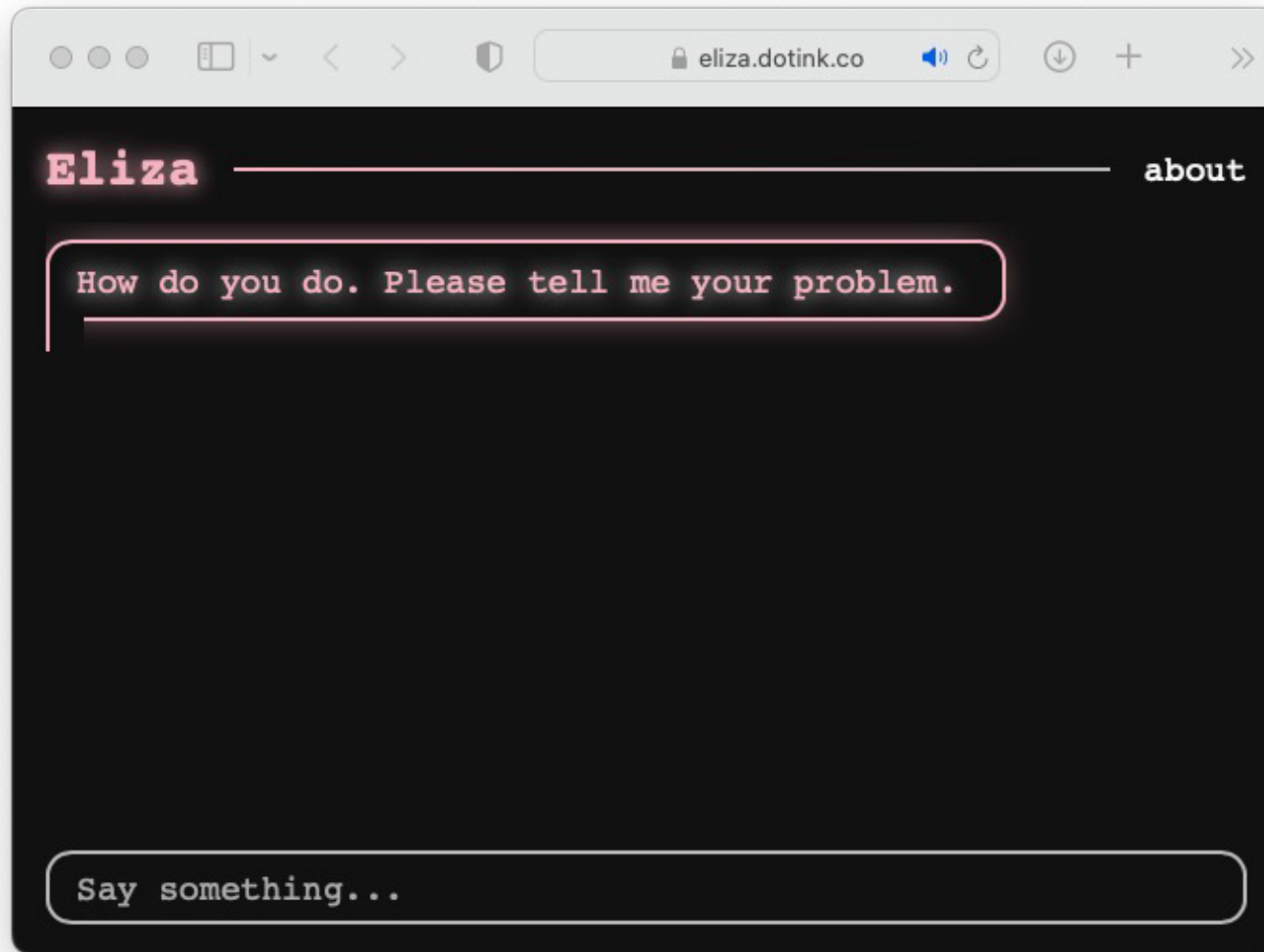
Sales of previously owned homes dipped 5.9% in July from the previous month. That was the weakest pace of sales since November 2015, excluding the three-month pandemic-related drop in the spring of 2020. Home-building is also drying up, and mortgage applications are falling as more buyers keep to the sidelines.

<https://huggingface.co/facebook/bart-large-cnn>

(translating <https://www.wsj.com/articles/existing-home-sales-prices-housing-market-july-2022-11660774574>)

Review/Intro: Some basic NLP applications/tools

Ex 3: Eliza, an AI-based psychotherapist



Review/Intro: Some basic NLP applications/tools

Ex 3: Eliza, an AI-based psychotherapist

Q: When do you think Eliza was created?

pre-chatGPT
(~5+ years ago)

10+ years

Q: Is Eliza an AI program?

yes

30+ years

Q: Does Eliza use Deep Learning?

not

Review/Intro: Basic NLP Applications/Tools

Ex 3: Eliza, an AI-based psychotherapist

```
# a translation table used to convert things you say into things the
# computer says back, e.g. "I am" --> "you are"

from nltk.chat.util import Chat, reflections

# a table of response pairs, where each pair consists of a
# regular expression, and a list of possible responses,
# with group-macros labelled as %1, %2.

pairs = (
    (
        r"I need (.*)"
        (
            "Why do you need %1?",
            "Would it really help you to get %1?",
            "Are you sure you need %1?",
        ),
    ),
    (
        r"Why don't you (.*)",
        (
            "Do you really think I don't %1?",
            "Perhaps eventually I will %1.",
            "Do you really want me to %1?",
        ),
    ),
),
```

NLP: Starting from scratch w/ a language problem

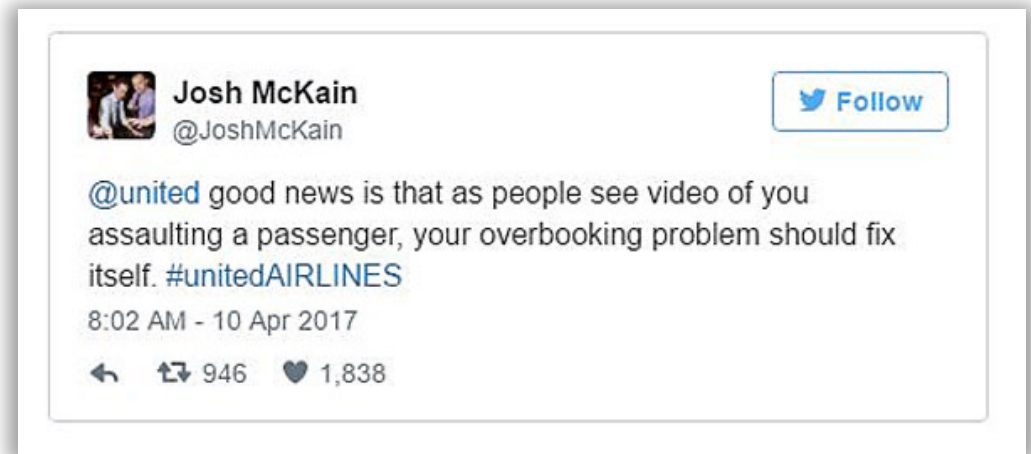
Problem: You work as a data scientist for a major airline company. You need to understand what customers/passengers are saying about the company on social media. Specifically, you will carry out **sentiment analysis** on all tweets directed to your company's Twitter handle.

(note: **sentiment analysis** is a special type of **text classification**)

NLP: Starting from scratch w/ a language problem

Problem: You work as a data scientist for a major airline company. You need to understand what customers/passengers are saying about the company on social media. Specifically, you will carry out **sentiment analysis** on all tweets directed to your company's Twitter handle.

Oftentimes it is so that the tweet can be passed on to the company's customer service so that they can reach out to the customer/passenger.



NLP: Starting from scratch w/ a language problem

Problem: You work as a data scientist for a major airline company. You need to understand what customers/passengers are saying about the company on social media. Specifically, you will carry out **sentiment analysis** on all tweets directed to your company's Twitter handle.

Other times it may be because there is a good PR/marketing opportunity.



NLP: Starting from scratch w/ a language problem

Problem: You work as a data scientist for a major airline company and need to create a **sentiment analysis** model to be run on all tweets directed to your company's Twitter handle. A tweet can be considered as either a) **positive**, b) **negative**, or c) **neutral**.

(Training) Dataset: Historical tweets that have been labeled (by humans)

	sentiment	text
0	positive	@JetBlue @JayVlg I like the inflight snacks! I'm flying with you guys on 2/28! #JVMChat
1	positive	@VirginAmerica thanks guys! Sweet route over the Rockies #airplanemodewason
2	negative	@US Airways Your exchange/credit policies are worthless and shadier than the White House. Dissatisfied to the nines right now.
3	negative	@US Airways but in the meantime I'll be sleeping on a park bench on Dadeland St. Thanks guys!
4	negative	@VirginAmerica hold times at call center are a bit much
5	negative	@US Airways not moving we are in the tarmac delayed for some unknown reason. I'll keep you posted
6	neutral	@JetBlue What about if I booked it through Orbitz? My email is correct, but there's a middle party.
7	negative	@United 2nd flight also delayed no pilots! But they boarded so we can just sit here! #scheduling
8	negative	@AmericanAir after 50 minutes on hold, and another 30 minutes on the call yes. Going to be pushing it to get to the airport on time now
9	positive	@JetBlue flight 117. proud to fly Jet Blue!

NLP: Starting from scratch w/ a language problem

(Training) Dataset: Historical tweets that have been labeled (by humans)

	sentiment	text
0	positive	@JetBlue @JayVlg I like the inflight snacks! I'm flying with you guys on 2/28! #JVMChat
1	positive	@VirginAmerica thanks guys! Sweet route over the Rockies #airplanemodewason
2	negative	@US Airways Your exchange/credit policies are worthless and shadier than the White House. Dissatisfied to the nines right now.
3	negative	@US Airways but in the meantime I'll be sleeping on a park bench on Dadeland St. Thanks guys!
4	negative	@VirginAmerica hold times at call center are a bit much
5	negative	@US Airways not moving we are in the tarmac delayed for some unknown reason. I'll keep you posted
6	neutral	@JetBlue What about if I booked it through Orbitz? My email is correct, but there's a middle party.
7	negative	@United 2nd flight also delayed no pilots! But they boarded so we can just sit here! #scheduling
8	negative	.@AmericanAir after 50 minutes on hold, and another 30 minutes on the call yes. Going to be pushing it to get to the airport on time now
9	positive	@JetBlue flight 117. proud to fly Jet Blue!

Note that in the future we won't have labels. Only this dataset has the labels, and we need to learn how to infer positive/negative/neutral using only this data.

NLP: Starting from scratch w/ a language problem

(Training) Dataset: Historical tweets that have been labeled (by humans)

	sentiment	text
0	positive	@JetBlue @JayVlg I like the inflight snacks! I'm flying with you guys on 2/28! #JVMChat
1	positive	@VirginAmerica thanks guys! Sweet route over the Rockies #airplanemodewason
2	negative	@US Airways Your exchange/credit policies are worthless and shadier than the White House. Dissatisfied to the nines right now.
3	negative	@US Airways but in the meantime I'll be sleeping on a park bench on dadeland st. Thanks guys!
4	negative	@VirginAmerica hold times at call center are a bit much
5	negative	@US Airways not moving we are in the tarmac delayed for some unknown reason. I'll keep you posted

Q: How do we start? What is a simple 'model' we could use?

associate words ~~must~~ used
in positive, negative, neutral
tweets

Quick Python Sales Pitch

Python: Background

Created in 1991 by Guido van Rossum, goals for Python were to be:

- ▶ easy and intuitive language just as powerful as major competitors
- ▶ open source so that anyone can contribute to its development
- ▶ allow for code that is as understandable as plain English
- ▶ suitable for everyday tasks, allowing for short development times

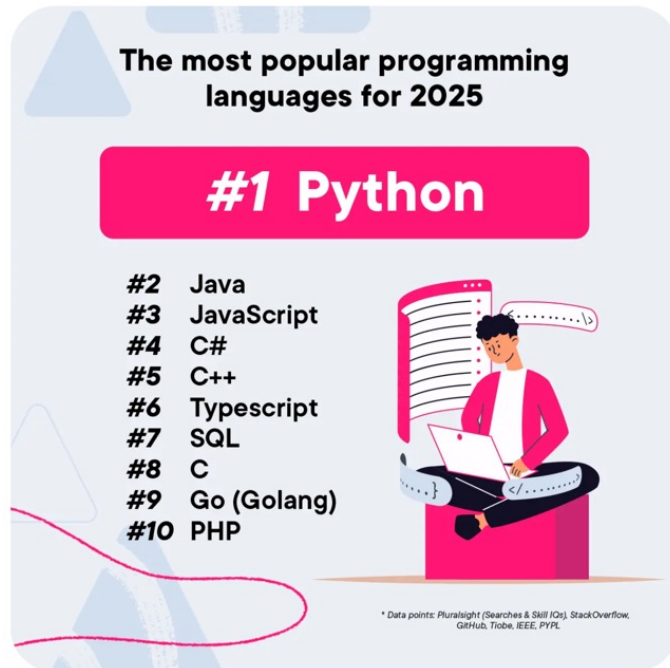
Python characteristics/properties:

- ▶ high-level, general-purpose language w/ emphasis on readability
- ▶ interpreted, dynamically typed, and garbage-collected
- ▶ comprehensive standard library (w/ extensive community)
- ▶ emphasizes readability with its notable use of whitespace
- ▶ object-oriented helps enable clear, logical code for small and large-scale projects (but also supports other paradigms)
- ▶ **de facto language for machine learning and data science!**

Python: Widely used and sought by employers

Ranked near top of all programming language since mid 00's

- ▶ top 10 of Tiobe index since 2004
- ▶ #1 language for Data Science / ML
- ▶ often top on in-demand lists, languages-to-learn lists, etc. (although, admittedly, not the best for Web Dev, Systems, etc.)



Contents

1. Python: The Versatile Vanguard
2. C++: The High-Performance Heavyweight
3. Java: The Enterprise Backbone
4. C#: The Microsoft Ecosystem VIP
5. JavaScript: The Evolving Giant
6. Go (Golang): The Rising Star of Cloud Infrastructure
7. Rust: The Future of Systems Programming
8. Ruby: The Startup Accelerator
9. Swift: The iOS Developer
10. Kotlin: The Java Modernizer

Why Python

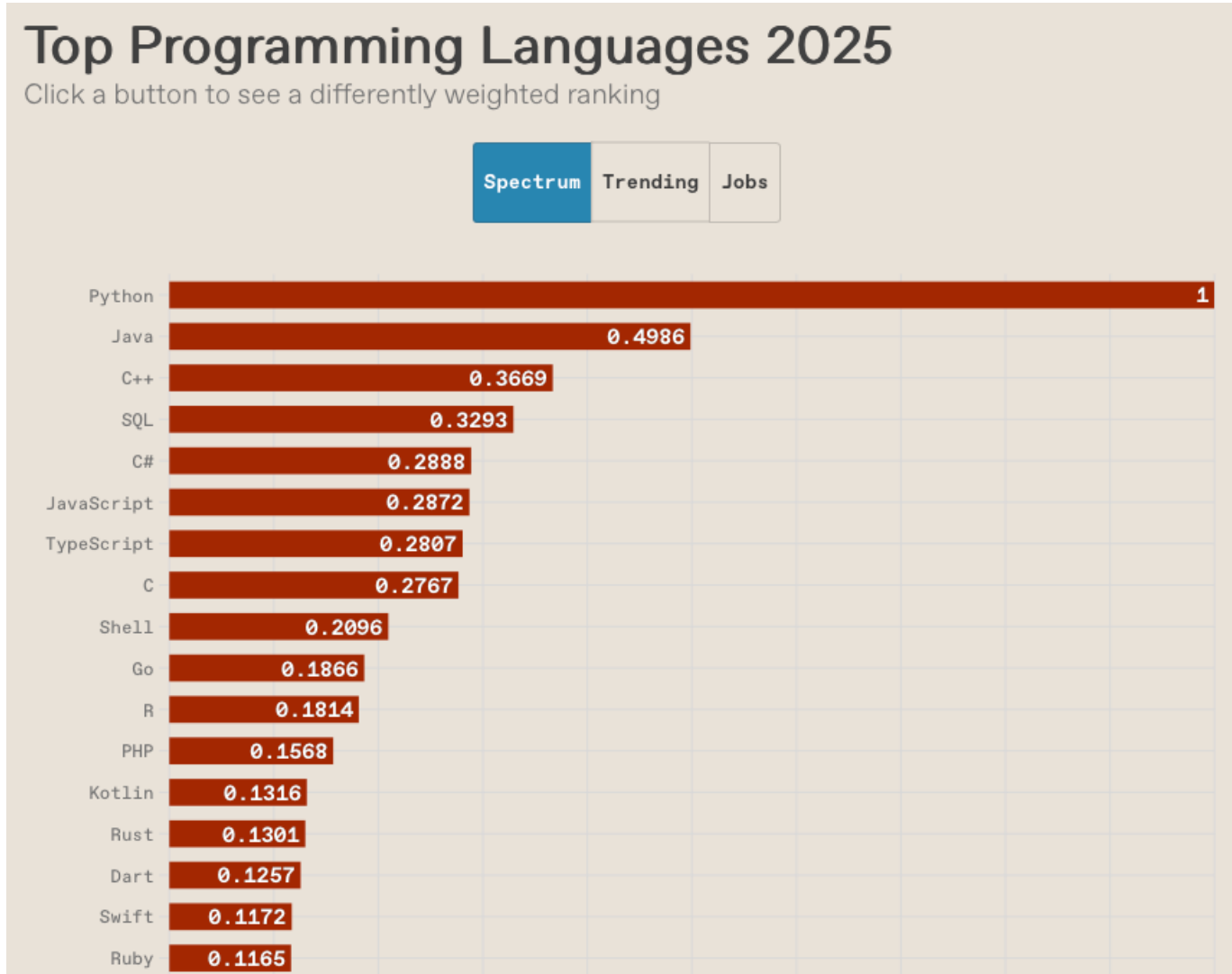
- **AI and Machine Learning:** With AI projected to contribute **\$15.7 trillion** to the global economy by 2030, Python's role in this field is undeniable. Its easy to learn syntax and powerful libraries like TensorFlow and PyTorch make it the go-to language for AI development. Whether you're building complex neural networks or implementing natural language processing, Python provides the tools you need.
- **Data Science and Analytics:** In a world awash with data, Python is an essential tool for making sense of it all. Its data manipulation and visualization libraries have revolutionized how we handle big data. From predictive analytics to **intricate data visualizations**, Python continues to be the language of choice for data scientists and analysts alike.
- **Web Development:** Python's reach extends beyond data and AI. Frameworks like Django and Flask push Python into the web development arena, especially for data-heavy applications. If you're looking to build scalable web apps that can handle complex data processing, Python offers a powerful solution.

<https://www.tiobe.com/tiobe-index/>

<https://spectrum.ieee.org/top-programming-languages-2025>

<https://survey.stackoverflow.co/2025/technology#most-popular-technologies-language>

Python: Widely used and sought by employers



<https://www.tiobe.com/tiobe-index/>

<https://spectrum.ieee.org/top-programming-languages-2025>

<https://survey.stackoverflow.co/2025/technology#most-popular-technologies-language>

Python: Widely used and sought by employers



<https://www.tiobe.com/tiobe-index/>

<https://spectrum.ieee.org/top-programming-languages-2025>

<https://survey.stackoverflow.co/2025/technology#most-popular-technologies-language>

Python: Installation and Use

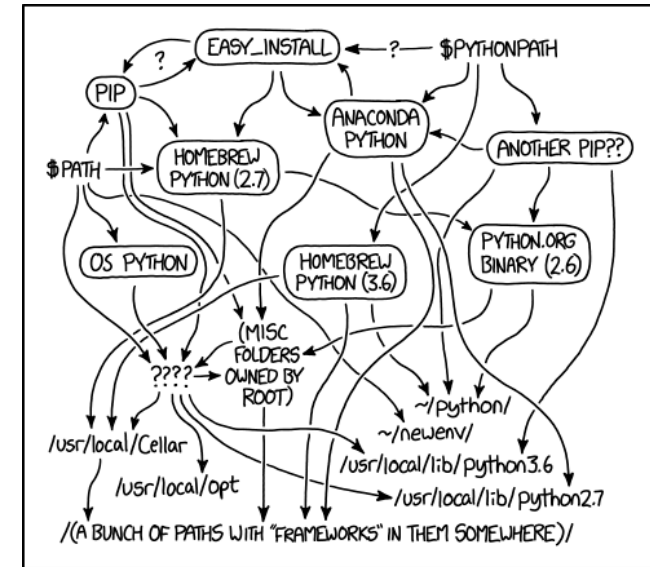
Installing Python (optional – you don't need to do so for this class!)

- ▶ Basic install: python.org
- ▶ Preferred install (for ML and DS): [anaconda](https://anaconda.org) common ML/DS modules)

Several ways to run Python:

- ▶ at the terminal w/ python or ipython
- ▶ text editor + terminal e.g. vim + python
- ▶ IDE / IDLE, e.g. VS Code, PyCharm
- ▶ interactive/R&D IDE

e.g. Jupyter notebooks (locally or remote on [Colab](https://colab.research.google.com), [Kaggle](https://www.kaggle.com), etc.)



MY PYTHON ENVIRONMENT HAS BECOME SO DEGRADED
THAT MY LAPTOP HAS BEEN DECLARED A SUPERFUND SITE.

Python: Installation and Use

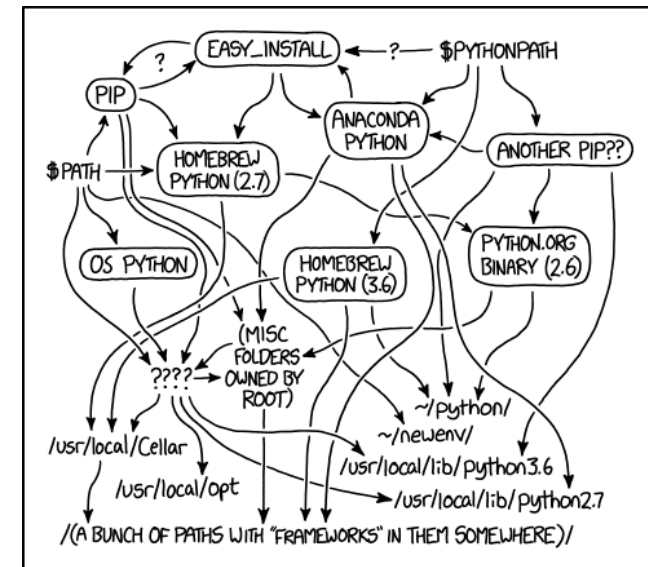
Installing Python (optional – you don't need to do so for this class!)

- ▶ Basic install: python.org
- ▶ Preferred install (for ML and DS): [anaconda](https://anaconda.org) common ML/DS modules

Several ways to run Python:

- ▶ at the terminal w/ python or ipython
- ▶ text editor + terminal e.g. vim + python
- ▶ IDE / IDLE, e.g. VS Code, PyCharm
- ▶ interactive/R&D IDE

e.g. Jupyter notebooks (locally or remote on [Colab](https://colab.research.google.com), [Kaggle](https://www.kaggle.com), etc.)



Note that for this semester in this class:

- ▶ although you can install and run Python locally,
- ▶ you do not need to install/run Python locally (nor do you need a high-end GPU)

Python: General Resources

Not mandatory, but a good idea to check out some Python tutorials (particularly if you're new to Python):

- <https://www.learnpython.org>
- <https://pythonprogramminglanguage.com>
- [Socratica Python videos for beginners](#)

Course Software: Other Modules/Tools

We'll also be using several other tools and packages that extend Python's functionality

These are:

- pandas
- numpy
- matplotlib
- scikit learn
- jupyter notebooks (locally or on Colab/Kaggle/Gradient)
- pytorch
- transformers (by huggingface)

Course Software: Pandas

Pandas: a Python module to work with datasets

pandas DataFrame object is a Python version of an Excel spreadsheet:

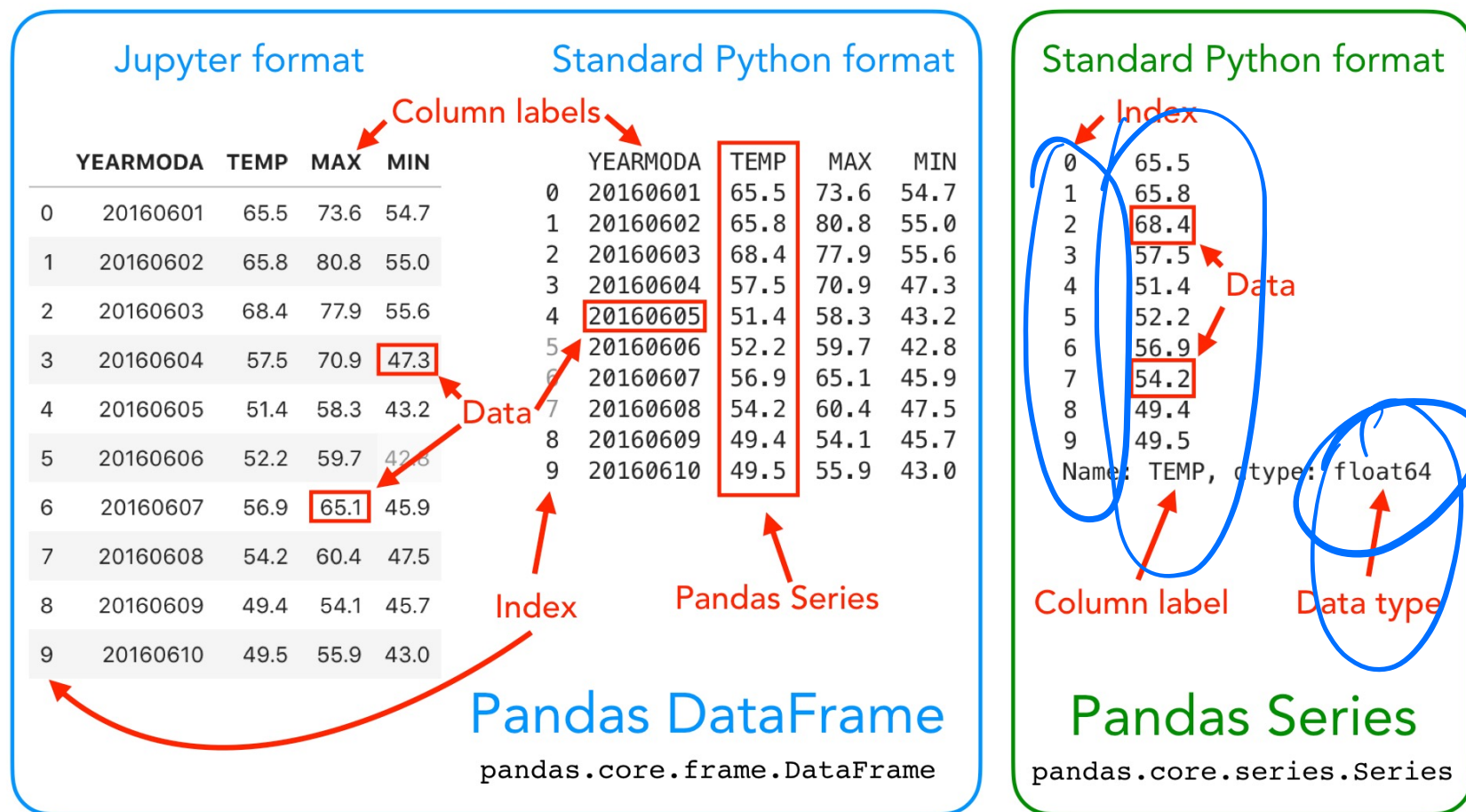
The diagram illustrates a pandas DataFrame as a table. The columns are labeled *Name*, *Team*, *Number*, *Position*, and *Age*. The rows are indexed from 0 to 6. Annotations include: blue circles around the *Name* and *Team* headers; blue arrows pointing from the *Columns* label to each header; orange arrows pointing from the *Rows* label to each row index; a blue oval around the *Team* column; a pink oval around the *Number* column; and a pink oval around the *Position* column. A pink bracket labeled *Data* spans the data cells of the *Number*, *Position*, and *Age* columns for rows 2 through 6. The MSU logo is in the bottom right corner of the diagram.

	<i>Name</i>	<i>Team</i>	<i>Number</i>	<i>Position</i>	<i>Age</i>
0	Avery Bradley	Boston Celtics	0.0	PG	25.0
1	John Holland	Boston Celtics	30.0	SG	27.0
2	Jonas Jerebko	Boston Celtics	8.0	PF	29.0
3	Jordan Mickey	Boston Celtics	NaN	PF	21.0
4	Terry Rozier	Boston Celtics	12.0	PG	22.0
5	Jared Sullinger	Boston Celtics	7.0	C	NaN
6	Evan Turner	Boston Celtics	11.0	SG	27.0

Course Software: Pandas

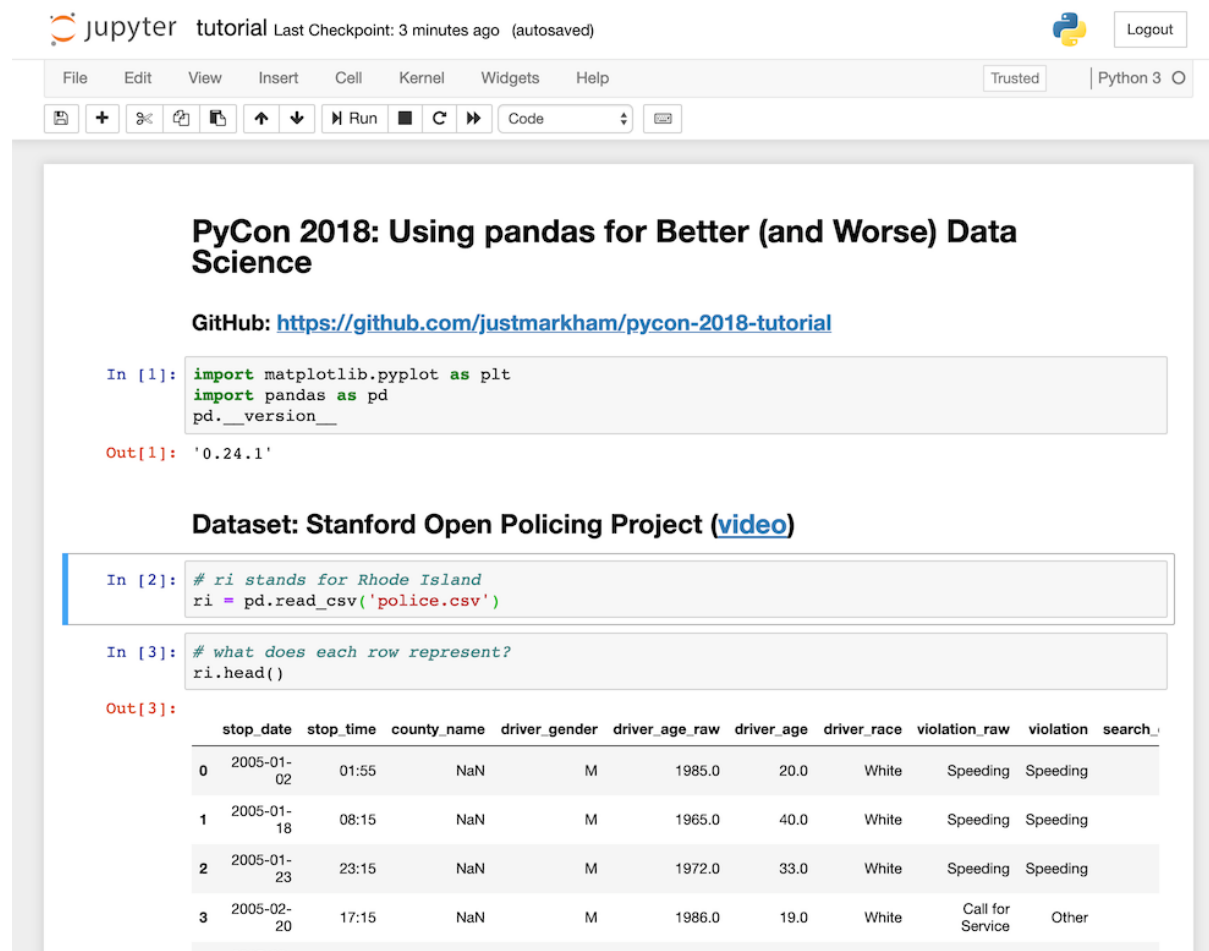
Pandas: a Python module to work with datasets

Two main data types implemented by Pandas:



Course Software: Jupyter Notebooks

Jupyter: *the* programming environment for data scientists
(note: Colab/Kaggle provide exact same functionality)



jupyter tutorial Last Checkpoint: 3 minutes ago (autosaved)  Logout

File Edit View Insert Cell Kernel Widgets Help Trusted Python 3

PyCon 2018: Using pandas for Better (and Worse) Data Science

GitHub: <https://github.com/justmarkham/pycon-2018-tutorial>

```
In [1]: import matplotlib.pyplot as plt
import pandas as pd
pd.__version__
```

```
Out[1]: '0.24.1'
```

Dataset: Stanford Open Policing Project ([video](#))

```
In [2]: # ri stands for Rhode Island
ri = pd.read_csv('police.csv')
```

```
In [3]: # what does each row represent?
ri.head()
```

```
Out[3]:
```

	stop_date	stop_time	county_name	driver_gender	driver_age_raw	driver_age	driver_race	violation_raw	violation	search_
0	2005-01-02	01:55	NaN	M	1985.0	20.0	White	Speeding	Speeding	
1	2005-01-18	08:15	NaN	M	1965.0	40.0	White	Speeding	Speeding	
2	2005-01-23	23:15	NaN	M	1972.0	33.0	White	Speeding	Speeding	
3	2005-02-20	17:15	NaN	M	1986.0	19.0	White	Call for Service	Other	

Course Software: Software Requirements

To reiterate from earlier slides...

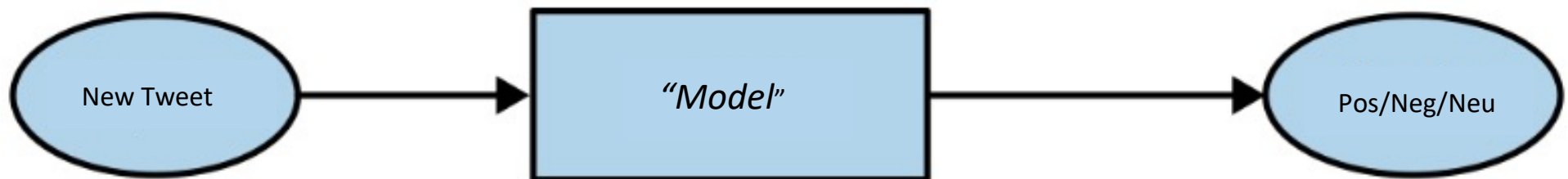
- yes, you can install and run Python locally if you want, but
- no, you don't need to install anything, and
- no, you don't need a high-end GPU

NLP: Starting from scratch w/ a language problem

Back to our Airline Tweet dataset...

	sentiment	text
0	positive	@JetBlue @JayVig I like the Inflight snacks! I'm flying with you guys on 2/28! #JVMChat
1	positive	@VirginAmerica thanks guys! Sweet route over the Rockies #airplanemodewason
2	negative	@US Airways Your exchange/credit policies are worthless and shadier than the White House. Dissatisfied to the nines right now.
3	negative	@US Airways but in the meantime I'll be sleeping on a park bench on dadeland st. Thanks guys!
4	negative	@VirginAmerica hold times at call center are a bit much
5	negative	@US Airways not moving we are in the tarmac delayed for some unknown reason. I'll keep you posted
6	neutral	@JetBlue What about if I booked it through Orbitz? My email is correct, but there's a middle party.
7	negative	@united 2nd flight also delayed no pilots! But they boarded is so we can just sit here! #scheduling

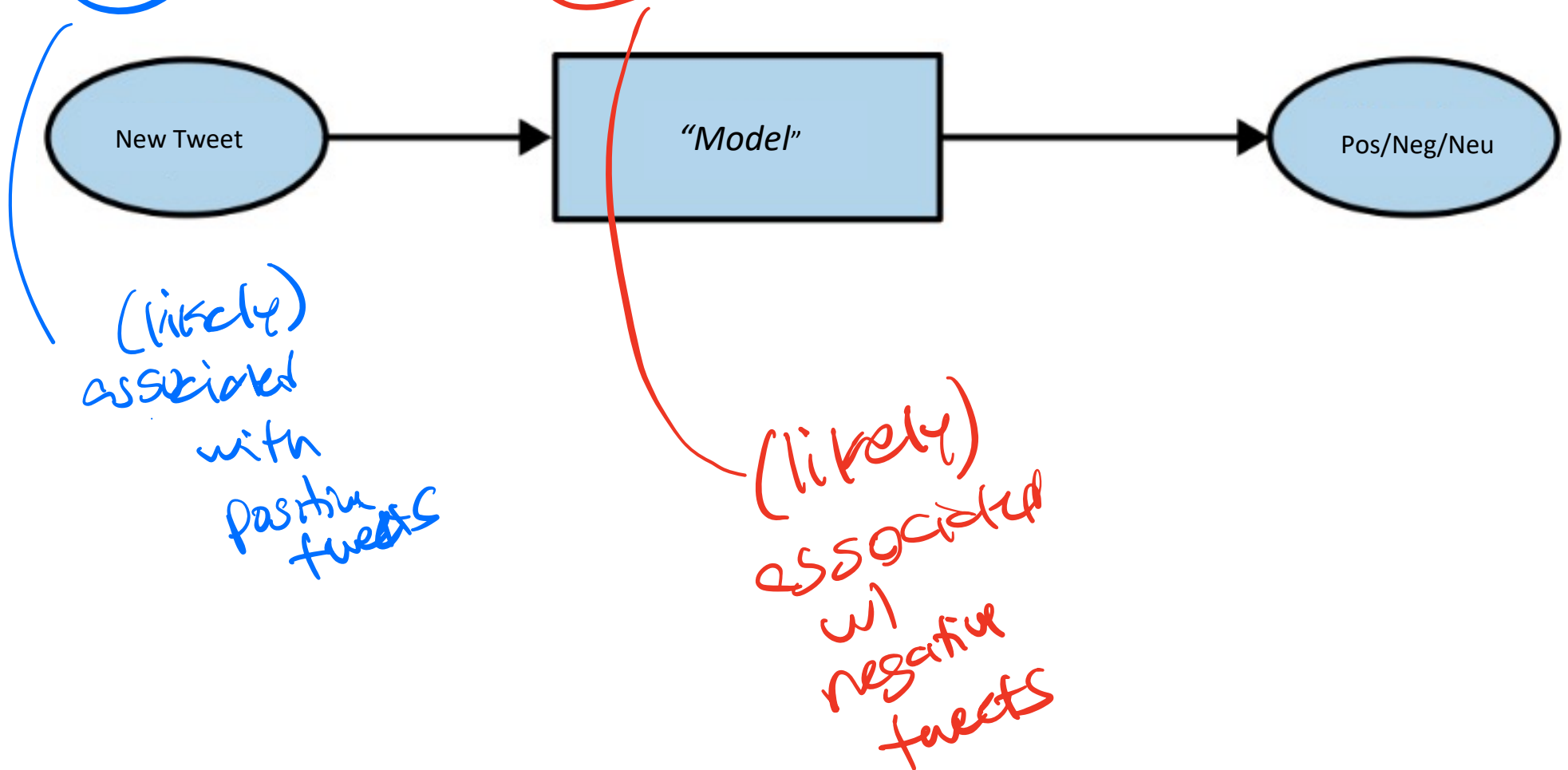
Q: What is a simple 'model' to predict if a new tweet is positive, negative, or neutral?



NLP: Starting from scratch w/ a language problem

New tweet:

"Thanks @United, another cancellation, another night sleeping in the airport"



NLP: Starting from scratch w/ a language problem

Problem: You work as a data scientist for a major airline company and need to create a **sentiment analysis** model to be run on all tweets directed to your company's Twitter handle. A tweet can be considered as either a) **positive**, b) **negative**, or c) **neutral**.

(Training) Dataset: 10k Historical tweets that have been labeled by hand

sentiment		text
0	positive	@JetBlue @JayVig I like the inflight snacks! I'm flying with you guys on 2/28! #JVMChat
1	positive	@VirginAmerica thanks guys! Sweet route over the Rockies #airplanemodewason
2	negative	@USAirways Your exchange/credit policies are worthless and shadier than the White House. Dissatisfied to the nines right now.
3	negative	@USAirways but in the meantime I'll be sleeping on a park bench on dadeland st. Thanks guys!
4	negative	@VirginAmerica hold times at call center are a bit much
5	negative	@USAirways not moving we are in the tarmac delayed for some unknown reason. I'll keep you posted
6	neutral	@JetBlue What about if I booked it through Orbitz? My email is correct, but there's a middle party.
7	negative	@united 2nd flight also delayed no pilots! But they boarded is so we can just sit here! #scheduling
8	negative	.@AmericanAir after 50 minutes on hold, and another 30 minutes on the call yes. Going to be pushing it to get to the airport on time now
9	positive	@JetBlue flight 117 proud to fly Jet Blue!

NLP: Starting from scratch w/ a language problem

One Solution: Count the frequency of each word in the full dataset but separately for positive, negative, neutral tweets. We'll find a lot of stop words ☹️

```
list(vocab_pos.items())[:20]
```

```
[('the', 646),  
 ('to', 601),  
 ('for', 460),  
 ('you', 421),  
 ('i', 383),  
 ('@southwestair', 381),  
 ('a', 350),  
 ('@jetblue', 340),  
 ('@united', 330),  
 ('thank', 320),  
 ('and', 284),  
 ('thanks', 263),  
 ('my', 245),  
 ('@americanair', 229),  
 ('in', 216),  
 ('on', 215),  
 ('flight', 209),  
 ('of', 173),  
 ('was', 160),  
 ('your', 160)]
```

```
list(vocab_neg.items())[:20]
```

```
[('to', 4237),  
 ('the', 2898),  
 ('i', 2519),  
 ('a', 2296),  
 ('and', 2049),  
 ('on', 1966),  
 ('for', 1930),  
 ('@united', 1828),  
 ('my', 1699),  
 ('flight', 1697),  
 ('you', 1694),  
 ('@usairways', 1671),  
 ('is', 1483),  
 ('@americanair', 1467),  
 ('in', 1219),  
 ('of', 1100),  
 ('your', 957),  
 ('not', 923),  
 ('no', 896),  
 ('have', 855)]
```

```
list(vocab_neu.items())[:20]
```

```
[('to', 1043),  
 ('i', 723),  
 ('the', 573),  
 ('a', 503),  
 ('on', 415),  
 ('@southwestair', 414),  
 ('@united', 411),  
 ('you', 395),  
 ('for', 380),  
 ('@jetblue', 365),  
 ('my', 338),  
 ('flight', 337),  
 ('@americanair', 293),  
 ('is', 291),  
 ('can', 278),  
 ('in', 272),  
 ('and', 254),  
 ('@usairways', 233),  
 ('of', 217),  
 ('from', 215)]
```

NLP: Starting from scratch w/ a language problem

One Solution (cont): So, let's first look at the most frequent words in the entire dataset (regardless of label value)...

```
vocab_sorted = dict(sorted(vocab.items(), key=lambda item: item[1], reverse=True))  
list(vocab_sorted.items())[:25]
```

```
[('to', 5881),  
 ('the', 4117),  
 ('i', 3625),  
 ('a', 3149),  
 ('for', 2770),  
 ('on', 2596),  
 ('and', 2587),  
 ('@united', 2569),  
 ('you', 2510),  
 ('my', 2282),  
 ('flight', 2243),  
 ('@usairways', 2061),  
 ('@americanair', 1989),  
 ('is', 1921),  
 ('in', 1707),  
 ('@southwestair', 1619),  
 ('of', 1490),  
 ('@jetblue', 1352),  
 ('your', 1208),  
 ('have', 1138),  
 ('was', 1100),  
 ('with', 1055),  
 ('me', 1053),  
 ('it', 1051),  
 ('not', 1047)]
```

NLP: Starting from scratch w/ a language problem

One Solution (cont): So, let's first look at the most frequent words in the entire dataset (regardless of label value)... and we removed the top $k = 100$ of these from the sets of positive, negative, and neutral words:

```
list(vocab_pos.items())[:25]
```

```
[('you!', 93),  
 ('thanks', 68),  
 ('.', 61),  
 ('best', 59),  
 ('appreciate', 43),  
 ('thanks.', 36),  
 ('amazing', 33),  
 ('awesome', 31),  
 ('i'll', 28),  
 ('nice', 26),  
 ('keep', 25),  
 ('follow', 24),  
 ('well', 23),  
 ('better', 23),  
 ('work', 22),  
 ('attendant', 22),  
 ('making', 21),  
 ('forward', 20),  
 ('always', 20),  
 ('sure', 20),  
 ('sent', 20),  
 ('trip', 20),  
 ('care', 19),  
 ('able', 19),  
 ('thx', 19)]
```

```
list(vocab_neg.items())[:25]
```

```
[('won't', 107),  
 ('bad', 103),  
 ('days', 102),  
 ('since', 101),  
 ('min', 99),  
 ('miss', 97),  
 ('where', 95),  
 ('find', 94),  
 ('doesn't', 89),  
 ('line', 89),  
 ('give', 88),  
 ('called', 86),  
 ('put', 86),  
 ('missed', 86),  
 ('tried', 85),  
 ('1', 83),  
 ('customers', 83),  
 ('work', 82),  
 ('too', 82),  
 ('5', 82),  
 ('same', 82),  
 ('pay', 80),  
 ('into', 80),  
 ('does', 79),  
 ('long', 79)]
```

```
list(vocab_neu.items())[:25]
```

```
[('fleet's', 68),  
 ('fleek.', 68),  
 ('@jetblue:', 52),  
 ('tomorrow', 38),  
 ('chance', 34),  
 ('follow', 32),  
 ('#destinationdragons', 31),  
 ('does', 31),  
 ('rt', 31),  
 ('booked', 29),  
 ('passengers', 29),  
 ('use', 29),  
 ('tickets', 29),  
 ('via', 27),  
 ('book', 27),  
 ('sent', 27),  
 ('what's', 27),  
 ('think', 26),  
 ('booking', 26),  
 ('ceo', 25),  
 ('into', 24),  
 ('aa', 24),  
 ('start', 22),  
 ('problems', 22),  
 ('send', 21)]
```

NLP: Starting from scratch w/ a language problem

One Solution (cont): To classify a new tweet, let's count how many words from the positive set, negative set, and neutral set it has. The label corresponding to the set with the highest number of words, is the label we should predict.

```
[62]: tweet2classify_i = 5555
      tweet2classify = df.iloc[tweet2classify_i,:]['tokens']
      df.iloc[tweet2classify_i,:]
```

```
[62]... sentiment
      negative
      text      @united no- we are boarding- but why can't your agents, on the phone, taking care of 1K travell
ers, link reservations?!?!
      tokens    [@united, no-, we, are, boarding-, but, why, can't, your, agents,, on, the, phone,, taking, care, of, 1k, traveller
s,, link, reservations?!?!]
      Name: 5555, dtype: object
```

```
[63]: pos = 0
      neg = 0
      neu = 0
      for tok in tweet2classify:
          if tok in classifier_tokens['positive']:
              pos += 1
          elif tok in classifier_tokens['negative']:
              neg += 1
          elif tok in classifier_tokens['neutral']:
              neu += 1

      print(f"pos: {pos}    neg: {neg}    neu: {neu}")
```

```
pos: 4    neg: 6    neu: 0
```

NLP: Starting from scratch w/ a language problem

One Solution (cont): To see how the model worked, overall, we can classify all 10k tweets using this approach and compared the predicted labels to the actual labels. Overall accuracy: ~38% (using k =100)

```
def predict_tweet_sentiment(tweet_tokens):
    pos = 0
    neg = 0
    neu = 0
    for tok in tweet_tokens:
        if tok in classifier_tokens['positive']:
            pos += 1
        elif tok in classifier_tokens['negative']:
            neg += 1
        elif tok in classifier_tokens['neutral']:
            neu += 1
    if pos > neg and pos > neu:
        return "positive"
    elif neu > pos and neu > neg:
        return "neutral"
    else:
        return "negative"
```

```
df['predicted_sentiment'] = df['tokens'].apply(lambda x: predict_tweet_sentiment(x))
```

```
df.head(10)
```

	sentiment	text	tokens	predicted_sentiment
0	positive	@JetBlue @JayVig I like the inflight snacks! I'm flying with you guys on 2/28! #JVMChat	[@jetblue, @jayvig, i, like, the, inflight, snacks!, i'm, flying, with, you, guys, on, 2/28!, #jvmchat]	positive
1	positive	@VirginAmerica thanks guys! Sweet route over the Rockies #airplanemodewason	[@virginamerica, thanks, guys!, sweet, route, over, the, rockies, #airplanemodewason]	positive
2	negative	@USAirways Your exchange/credit policies are worthless and shadier than the White House. Dissatisfied to the nines right now.	[@usairways, your, exchange/credit, policies, are, worthless, and, shadier, than, the, white, house., dissatisfied, to, the, nines, right, now.]	negative
3	negative	@USAirways but Notebook editor cells sleeping on a park bench on dadeland st. Thanks guys!	[@usairways, but, in, the, meantime, i'll, be, sleeping, on, a, park, bench, on, dadeland, st., thanks, guys!]	negative
4	negative	@VirginAmerica hold times at call center are a bit much	[@virginamerica, hold, times, at, call, center, are, a, bit, much]	positive
5	negative	@USAirways not moving we are in the tarmac delayed for some unknown reason. I'll keep you posted	[@usairways, not, moving, we, are, in, the, tarmac, delayed, for, some, unknown, reason., i'll, keep, you, posted]	positive
6	neutral	@JetBlue What about if I booked it through Orbitz? My email is correct, but there's a middle party.	[@jetblue, what, about, if, i, booked, it, through, orbitz?, my, email, is, correct., but, there's, a, middle, party.]	negative
7	negative	@united 2nd flight also delayed no pilots! But they boarded is so we can just sit here! #scheduling	[@united, 2nd, flight, also, delayed, no, pilots!, but, they, boarded, is, so, we, can, just, sit, here!, #scheduling]	negative
8	negative	.@AmericanAir after 50 minutes on hold, and another 30 minutes on the call yes. Going to be pushing it to get to the airport on time now	[@americanair, after, 50, minutes, on, hold,, and, another, 30, minutes, on, the, call, yes., going, to, be, pushing, it, to, get, to, the, airport, on, time, now]	negative
9	positive	@JetBlue flight 117. proud to fly Jet Blue!	[@jetblue, flight, 117., proud, to, fly, jet, blue!]	positive

NLP: Starting from scratch w/ a language problem

One Solution (cont): To see how the model worked, overall, we can classify all 10k tweets using this approach and compared the predicted labels to the actual labels. Overall accuracy: ~38%

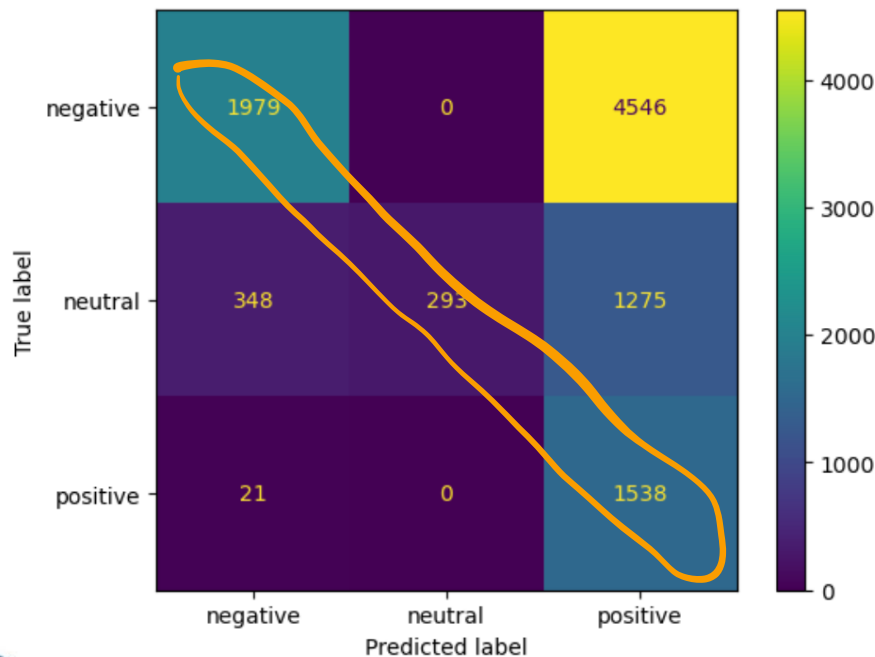
Can also use the Confusion Matrix to better understand model performance

```
from sklearn.metrics import confusion_matrix, ConfusionMatrixDisplay, accuracy_score
confusion_matrix(df['sentiment'], df['predicted_sentiment'])
disp = ConfusionMatrixDisplay(confusion_matrix(df['sentiment'], df['predicted_sentiment']))
disp.plot()
```

```
mod_accuracy = accuracy_score(df['sentiment'], df['predicted_sentiment'])
print(f"model accuracy: {mod_accuracy*100:.2f}%")
```

✓ 0.0s

model accuracy: 38.10%



Q: How do we calculate the accuracy from this?

$$\text{accuracy} = \frac{1979 + 293 + 1538}{10k} = 0.381$$

NLP: Starting from scratch w/ a language problem

Q1: Is the data good? Can the tokenization be improved?

Q2: How is our “model”? Can we improve it?

Q3: Is our modeling and model evaluation process okay?
How can it be improved?

Tokenization: Dealing with punctuation

Punctuation and special characters are easy to deal with, we just need to use

thanks! vs. thanks

.. ..
...

miss vs. missed
vs. missing

Tokenization: Dealing with punctuation

Punctuation and special characters are easy to deal with, we just need to use

regular expressions!

Ex: *

```
>>> text = 'That U.S.A. poster-print costs $12.40...'
>>> pattern = r'''(?x)      # set flag to allow verbose regexps
...    ([A-Z]\.)+          # abbreviations, e.g. U.S.A.
...    | \w+(-\w+)*         # words with optional internal hyphens
...    | \$?\d+(\.\d+)?%?    # currency and percentages, e.g. $12.40, 82%
...    | \.\.\.             # ellipsis
...    | [][.,;"'()?():-_' ] # these are separate tokens; includes ], [
...    '''
>>> nltk.regexp_tokenize(text, pattern)
['That', 'U.S.A.', 'poster-print', 'costs', '$12.40', '...']
```

Tokenization: Dealing with punctuation

Note that we'll typically rely on tools (e.g. nltk) to deal with basic tokenization issues such as this:



Input[1]

not - language toolkit

```
from nltk.tokenize import TweetTokenizer
tweet = "Snow White and the Seven Degrees #MakeAMovieCold@midnight:-)"
tokenizer = TweetTokenizer()
print(tokenizer.tokenize(tweet.lower()))
```

Output[1]

```
['snow', 'white', 'and', 'the', 'seven', 'degrees',
 '#makeamoviecold', '@midnight', ':-)']
```

Tokenization: Dealing with different word forms

Q: What about words that have the same meaning but are in the dataset in various inflected forms? (e.g. 'fly' vs 'flew' vs 'flies', 'flown')

Tokenization: Dealing with different word forms

Q: What about words that have the same meaning but are in the dataset in various inflected forms? (e.g. 'fly' vs 'flew' vs 'flies', 'flown')

Note: Agglutinative languages combine words to create new meaning, so how should these be tokenized?

Turkish	English
kork(-mak)	(to) fear
korku	fear
korkusuz	fearless
korkusuzlaş (-mak)	(to) become fearless
korkusuzlaşmış	One who has become fearless
korkusuzlaştır(-mak)	(to) make one fearless
korkusuzlaştırıl(-mak)	(to) be made fearless
korkusuzlaştırılmış	One who has been made fearless
korkusuzlaştırılabil(-mek)	(to) be able to be made fearless
korkusuzlaştırılabilir	One who will be able to be made fearless
korkusuzlaştırabileceklerimiz	Ones who we can make fearless
korkusuzlaştırabileceklerimizden	From the ones who we can make fearless
korkusuzlaştırabileceklerimizdenmiş	I gather that one is one of those we can make fearless
korkusuzlaştırabileceklerimizdenmişçesine	As if that one is one of those we can make fearless
korkusuzlaştırabileceklerimizdenmişçesineyken	when it seems like that one is one of those we can make fearless

Tokenization: Dealing with different word forms

Two main approaches to preprocessing text to deal with different word forms:

1. Lemmatization – attempts to identify the root form of a word and converts all inflected forms to this root form
2. Stemming – truncates words according to defined set of rules to get closer to a root form

Tokenization: Dealing with different word forms

Let's process the following words using lemmatization and stemming.

```
words = ["program", "programs", "programming", "programmers", "corpus", "corpora"]
```

Q: Which example below uses lemmatization (and which uses stemming)?

stemming

```
program : program
programs : program
programming : program
programmers : programm
corpus : corpu
corpora : corpora
```

lemmatization

```
program : program
programs : program
programming : programming
programmers : programmer
corpus : corpus
corpora : corpus
```

Tokenization: Dealing with different word forms

1. Lemmatization – attempts to identify the root form of a word and converts all inflected forms to this root form, generally requires an external database (e.g. Wordnet*), and sometimes even utilizes a language model for pos tagging
2. Stemming – truncates words according to defined set of rules to get closer to a root form, can be hardcoded based on common suffixes

```
from nltk.stem import WordNetLemmatizer
# also need to run following one time on your system
# (can even be outside of this notebook)
# import nltk
# nltk.download('wordnet')
# nltk.download('omw-1.4')
lemmatizer = WordNetLemmatizer()
```

```
for w in words:
    print(w, " : ", lemmatizer.lemmatize(w, pos="n"))
```

✓ 0.2s

```
program : program
programs : program
programming : programming
programmers : programmer
corpus : corpus
corpora : corpus
```

```
from nltk.stem import PorterStemmer
ps = PorterStemmer()
```

```
for w in words:
    print(w, " : ", ps.stem(w))
```

✓ 0.2s

```
program : program
programs : program
programming : program
programmers : programm
corpus : corpu
corpora : corpora
```

Tokenization: Dealing with different word forms

1. Lemmatization – attempts to identify the root form of a word and converts all inflected forms to this root form
2. Stemming – truncates words according to defined set of rules to get closer to a root form

Strength

Weakness

Tokenization: Summary

The purpose of **tokenizing** the data is to allow us to produce a clean set of words that we can use to then create a **vocabulary**

	sentiment	text	tokens_raw	tokens
5464	positive	@VirginAmerica first time flying Virgin, went to #SanFrancisco .Thanks for the smooth ride. Easily my new fav airline!	[@virginamerica, first, time, flying, virgin, ,, went, to, #sanfrancisco, ., thanks, for, the, smooth, ride, ., easily, my, new, fav, airline, !]	[@virginamerica, first, time, fly, virgin, ,, go, to, #sanfrancisco, ., thank, for, the, smooth, ride, ., easily, my, new, fav, airline, !]

Tokenization: Summary

The purpose of **tokenizing** the data is to allow us to produce a clean set of words that we can use to then create a **vocabulary**

	sentiment	text	tokens_raw	tokens
5464	positive	@VirginAmerica first time flying Virgin, went to #SanFrancisco .Thanks for the smooth ride. Easily my new fav airline!	[@virginamerica, first, time, flying, virgin, ,, went, to, #sanfrancisco, ., thanks, for, the, smooth, ride, ., easily, my, new, fav, airline, !]	[@virginamerica, first, time, fly, virgin, ,, go, to, #sanfrancisco, ., thank, for, the, smooth, ride, ., easily, my, new, fav, airline, !]

Definitions to know:

Vocabulary – the unique set of tokens (typically words) after dealing with punctuation, removing stop words, and lemmatizing/stemming content words

Stop words – articles and prepositions (e.g. 'the', 'by', etc.) that serve mostly a grammatical purpose, and which act like fillers to hold the content words

Content words – words that carry the meaning of a piece of text that we will keep to create the vocabulary

Types – unique tokens we keep to create the vocabulary

Tokenization: Summary

The purpose of **tokenizing** the data is to allow us to produce a clean set of words that we can use to then create a **vocabulary**

	sentiment	text	tokens_raw	tokens
5464	positive	@VirginAmerica first time flying Virgin, went to #SanFrancisco .Thanks for the smooth ride. Easily my new fav airline!	[@virginamerica, first, time, flying, virgin, ,, went, to, #sanfrancisco, ., thanks, for, the, smooth, ride, ., easily, my, new, fav, airline, !]	[@virginamerica, first, time, fly, virgin, ,, go, to, #sanfrancisco, ., thank, for, the, smooth, ride, ., easily, my, new, fav, airline, !]

Toy Example:

Dataset (or **corpus**):

Observation 1: *"Time flies like an arrow."*

Observation 2: *"Fruit flies like a banana."*

Vocabulary for this dataset:

{ time, fruit, flies, }

Back to our Airline Tweet Data/Model Questions

Q1: Is the data good? Can the tokenization be improved?

Q2: How is our “model”? Can we improve it?

Q3: Is our modeling and model evaluation process okay?
How can it be improved?

ML/Statistical Models: A Typical Example

The following dataset is a list of historical home sale prices and all of the dimensions, features, etc. for each home.

	MSSubClass	LotFrontage	LotArea	OverallQual	OverallCond	YearBuilt	...	GarageArea	OpenPorchSF	MoSold	YrSold	SalePrice
0	60	65.0	8450	7	5	2003	...	548	61	2	2008	208500
1	20	80.0	9600	6	8	1976	...	460	0	5	2007	181500
2	60	68.0	11250	7	5	2001	...	608	42	9	2008	223500
3	70	60.0	9550	7	5	1915	...	642	35	2	2006	140000
4	60	84.0	14260	8	5	2000	...	836	84	12	2008	250000

A company wants to use this data to train a model to be able to predict the sale price of a home well before it is sold (so that the company can purchase it at the optimal price to make a profit after flipping/renting it)

Q: How is this data different from our tweet dataset (after tokenizing)?

Q: What kind of model can be used with this data?

Vectorization: Turning Text Data into Numeric Data

Toy Example (pretend that we decided 'a' and 'an' are not stop words)

Dataset (or ~~corpus~~):

Observation 1: *"Time flies like an arrow."*

Observation 2: *"Fruit flies like a banana."*

Vocabulary for this dataset:

~~Dictionary~~ {time, fruit, flies, like, a, an, arrow, banana}

Q: How do we turn this vocabulary into numeric data?

Vectorization: Turning Text Data into Numeric Data

Toy Example (pretend that we decided 'a' and 'an' are not stop words)

Dataset (or corpus):

Observation 1: "Time flies like an arrow."

Observation 2: "Fruit flies like a banana."

Vocabulary for this dataset:

{time, fruit, flies, like, a, an, arrow, banana}

Q: How can we use this vocabulary to convert observations to numeric data?

A: One-hot Encoding

	time	fruit	flies	like	a	an	arrow	banana
1 time								
1 fruit								
1 flies								
1 like								
1 a								
1 an								
1 arrow								
1 banana								

Vectorization: Turning Text Data into Numeric Data

Toy Example (pretend that we decided 'a' and 'an' are not stop words)

Dataset (or corpus):

Observation 1: "Time flies like an arrow."

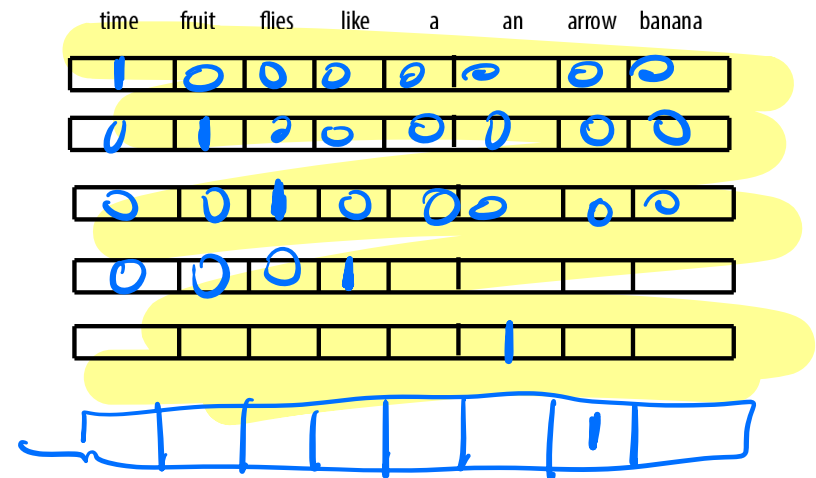
Observation 2: "Fruit flies like a banana."

Vocabulary for this dataset:

{time, fruit, flies, like, a, an, arrow, banana}

Q: What are the one-hot encoded vectors for Observation 1?

	time	fruit	flies	like	a	an	arrow	banana
1 time								
1 fruit								
1 flies								
1 like								
1 a								
1 an								
1 arrow								
1 banana								



Vectorization: Turning Text Data into Numeric Data

Q: How could Observation 1's one-hot vectors be used as input to a model? (e.g. linear regression, random forest, etc.)

Observation 1: *"Time flies like an arrow."*

time	fruit	flies	like	a	an	arrow	banana

Vectorization: Turning Text Data into Numeric Data

Dataset (or corpus):

Observation 1: "Time flies like an arrow."

Observation 2: "Fruit flies like a banana."

Oftentimes, to reduce input dimensions, each observation is **collapsed**

Collapsed One-hot Encoding: Use logical OR for each one-hot encoded type (or token) in the vocabulary to produce a single vector

Observation 1:

	time	fruit	flies	like	a	an	arrow	banana
1 time	1	0	0	0	0	0	0	0
1 fruit	0	1	0	0	0	0	0	0
1 flies	0	0	1	0	0	0	0	0
1 like	0	0	0	1	0	0	0	0
1 a	0	0	0	0	1	0	0	0
1 an	0	0	0	0	0	1	0	0
1 arrow	0	0	0	0	0	0	1	0
1 banana	0	0	0	0	0	0	0	1

Observation 1 collapsed:

	time	fruit	flies	like	a	an	arrow	banana
1	1	0	0	0	0	0	0	0
2	0	1	0	0	0	0	0	0
3	0	0	1	0	0	0	0	0
4	0	0	0	1	0	0	0	0
5	0	0	0	0	1	0	0	0
6	0	0	0	0	0	1	0	0
7	0	0	0	0	0	0	1	0
8	0	0	0	0	0	0	0	1

Vectorization: Turning Text Data into Numeric Data

Dataset (or corpus):

Observation 1: *"Time flies like an arrow like time."*

Observation 2: *"Fruit flies like a banana."*

Vocabulary for this dataset:

{time, fruit, flies, like, a, an, arrow, banana}

Q: What information does collapsed one-hot encoding miss out on for the above observations? (aside from the ordering of the words/tokens)

Vectorization: Turning Text Data into Numeric Data

Dataset (or corpus):

Observation 1: *"Time flies like an arrow like time."*

Observation 2: *"Fruit flies like a banana."*

Vocabulary for this dataset:

`{time, fruit, flies, like, a, an, arrow, banana}`

Term Frequency (TF): Sum one-hot encoded vector for each type (or token) in the vocabulary to produce a single vector

Q: What would vector for Observation 1 look like using term-frequency?

time fruit flies like a an arrow banana

Vectorization: Turning Text Data into Numeric Data

Dataset (or corpus):

Observation 1: *"Time flies like an arrow like time."*

Observation 2: *"Fruit flies like a banana."*

Vocabulary for this dataset:

{time, fruit, flies, like, a, an, arrow, banana}

Term Frequency-Inverse Document Frequency (TF-IDF): similar to TF representation but also gives more weight to words that are unique to a single document

In addition to $TF(w)$ for each word (and for each document), need to also calculate $IDF(w)$ for each word (across all documents)

$TF(w, D) =$

$IDF(w) =$

$TFIDF(w, D) = TF(w, D) * IDF(w)$

Vectorization: Turning Text Data into Numeric Data

Dataset (or corpus):

Observation 1: *"Time flies like an arrow like time."*

Observation 2: *"Fruit flies like a banana."*

Vocabulary for this dataset:

{time, fruit, flies, like, a, an, arrow, banana}

Term Frequency-Inverse Document Frequency (TF-IDF): similar to TF representation but also gives more weight to words that are unique to a single document

In addition to $TF(w)$ for each word (and for each document), need to also calculate $IDF(w)$ for each word (across all documents)

time fruit flies like a an arrow banana

Vectorization: Turning Text Data into Numeric Data

Term Frequency-Inverse Document Frequency (TF-IDF): similar to TF representation but also gives more weight to words that are unique to a single document

In practice, most tools to calculate TF-IDF also use smoothing and normalization

`sklearn.feature_extraction.text.TfidfTransformer`

```
class sklearn.feature_extraction.text.TfidfTransformer(*, norm='l2', use_idf=True, smooth_idf=True, sublinear_tf=False)
```

[\[source\]](#)

Transform a count matrix to a normalized tf or tf-idf representation.

Tf means term-frequency while tf-idf means term-frequency times inverse document-frequency. This is a common term weighting scheme in information retrieval, that has also found good use in document classification.

The goal of using tf-idf instead of the raw frequencies of occurrence of a token in a given document is to scale down the impact of tokens that occur very frequently in a given corpus and that are hence empirically less informative than features that occur in a small fraction of the training corpus.

The formula that is used to compute the tf-idf for a term t of a document d in a document set is $\text{tf-idf}(t, d) = \text{tf}(t, d) * \text{idf}(t)$, and the idf is computed as $\text{idf}(t) = \log [n / \text{df}(t)] + 1$ (if `smooth_idf=False`), where n is the total number of documents in the document set and $\text{df}(t)$ is the document frequency of t ; the document frequency is the number of documents in the document set that contain the term t . The effect of adding "1" to the idf in the equation above is that terms with zero idf , i.e., terms that occur in all documents in a training set, will not be entirely ignored. (Note that the idf formula above differs from the standard textbook notation that defines the idf as $\text{idf}(t) = \log [n / (\text{df}(t) + 1)]$).

Vectorization: Turning Text Data into Numeric Data

Term Frequency-Inverse Document Frequency (TF-IDF): similar to TF representation but also gives more weight to words that are unique to a single document

```
Ex: corpus = ['Time flies flies like an arrow.',  
             'Fruit flies like a banana.']  
  
from sklearn.feature_extraction.text import TfidfVectorizer  
import seaborn as sns  
  
tfidf_vectorizer = TfidfVectorizer()  
tfidf = tfidf_vectorizer.fit_transform(corpus).toarray()  
sns.heatmap(tfidf, annot=True, cbar=False, xticklabels=vocab,  
            yticklabels= ['Sentence 1', 'Sentence 2'])
```



Back to our Airline Tweet Data/Model Questions

Q1: Is the dataset good? Can it be improved?

Can it be converted to a structured and numeric format to be able to be used in other types of models?

Q2: How is our “model”? Can we improve it?

What other types of models could we use?

Q3: Is our modeling and model assessment process okay?

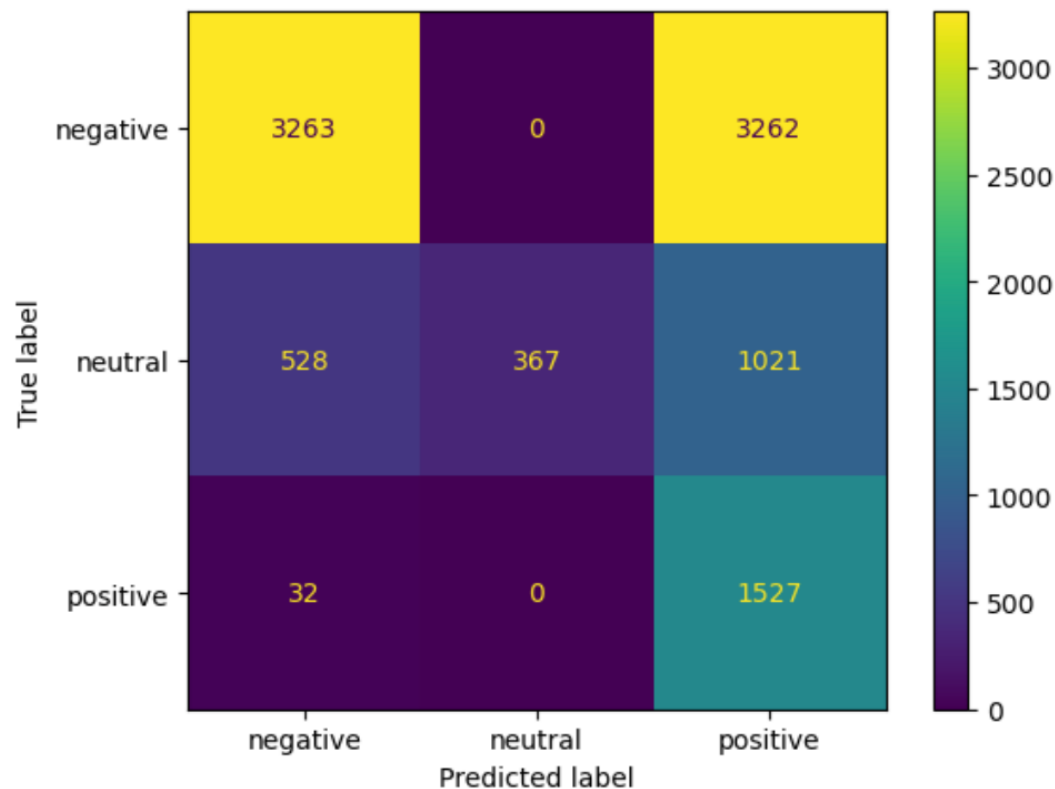
How can it be improved?

Model Assessment: Model Performance Metrics

Our Solution: Overall accuracy was ~38% and we looked at accuracy by label with the Confusion Matrix.

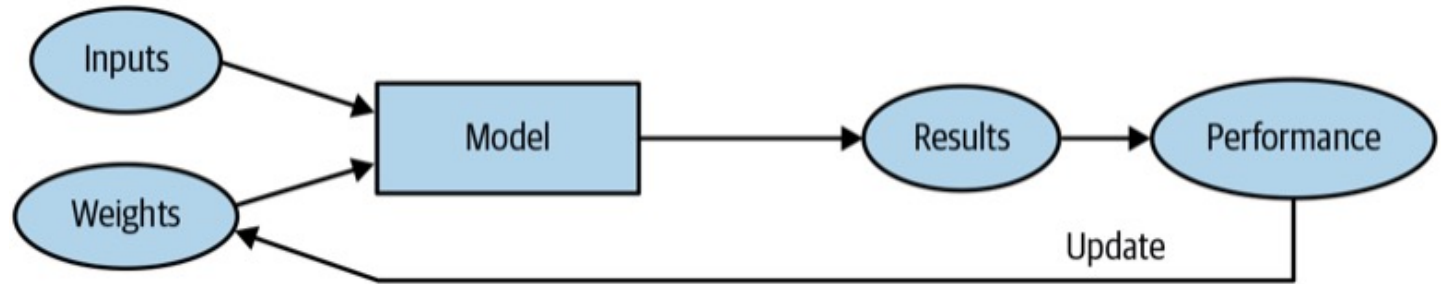
```
from sklearn.metrics import ConfusionMatrixDisplay
disp = ConfusionMatrixDisplay(confusion_matrix(df['sentiment'], df['predicted_sentiment']),
disp.plot()
```

<sklearn.metrics._plot.confusion_matrix.ConfusionMatrixDisplay at 0x7be6cbc5fc10>



Airline Tweets: Using a 'real' ML Model

Q: What would the inputs look like after using collapsed one-hot encoding? Or, using TF, or TF-IDF?



Today

- ▶ Course Overview
- ▶ ML/DL/AI Review
- ▶ Python Intro

To Do

- ▶ Self-introduction on Canvas (Jan 29th)
- ▶ Join MS Teams (code: w87cca**f**)
- ▶ Lab 1 (due Sat, Jan 31st)
- ▶ Kaggle 1 (due Mon, Feb 2nd)
- ▶ Optional tutorial/courses

- ▶ <https://www.learnpython.org/>
- ▶ <https://www.kaggle.com/learn/pandas>
- ▶ <https://www.kaggle.com/code/jhoward/jupyter-notebook-101>

Next Time (Thursday, Jan 29th) – No Class on Thur. nor next Tues.

- ▶ ML Overview/review – types of models, training, and metrics/performance
- ▶ Neural Networks



Jay Alammam
@JayAlammam

Software is eating the world.

Machine learning is eating software.



'ELIZA,' the world's 1st chatbot, was just resurrected from 60-year-old computer code

News By Kristina Killgrove published January 17, 2025

Researchers discovered long-lost computer code and used it to resurrect the early chatbot ELIZA.

<https://www.livescience.com/technology/eliza-the-worlds-1st-chatbot-was-just-resurrected-from-60-year-old-computer-code>